LibJuno Software Requirements Specification (SRS)

Table of Contents

1. Introduction . . . 6
   1.1. Purpose . . . 6
   1.2. Scope . . . 6
   1.3. Definitions and Acronyms . . . 6
   1.4. References . . . 7
2. Overall Description . . . 7
   2.1. Product Perspective . . . 7
   2.2. Design Constraints . . . 7
   2.3. Assumptions and Dependencies . . . 7
3. Specific Requirements . . . 7
   3.1. Module: APP . . . 7
      3.1.1. REQ-APP-001: Application Root Structure . . . 8
      3.1.2. REQ-APP-002: OnStart Lifecycle Hook . . . 8
      3.1.3. REQ-APP-003: OnProcess Lifecycle Hook . . . 8
      3.1.4. REQ-APP-004: OnExit Lifecycle Hook . . . 9
   3.2. Module: ARRAY . . . 9
      3.2.1. REQ-ARRAY-001: Array Initialization . . . 9
      3.2.2. REQ-ARRAY-002: Array SetAt . . . 10
      3.2.3. REQ-ARRAY-003: Array GetAt . . . 10
      3.2.4. REQ-ARRAY-004: Array RemoveAt . . . 10
      3.2.5. REQ-ARRAY-005: Array Bounds Checking . . . 11
      3.2.6. REQ-ARRAY-006: Array Vtable Validation . . . 11
      3.2.7. REQ-ARRAY-007: Array Capacity Validation . . . 12
   3.3. Module: CRC . . . 12
      3.3.1. REQ-CRC-001: CRC Streaming Update . . . 13
      3.3.2. REQ-CRC-002: CRC Null Safety . . . 13
      3.3.3. REQ-CRC-003: CRC Zero Length Safety . . . 14
      3.3.4. REQ-CRC-004: CRC-16 ARC . . . 14
      3.3.5. REQ-CRC-005: CRC-16 BinHex . . . 15
      3.3.6. REQ-CRC-006: CRC-16 CCITT . . . 15
      3.3.7. REQ-CRC-007: CRC-16 Kermit . . . 16
      3.3.8. REQ-CRC-008: CRC-32 CCITT32 . . . 16
      3.3.9. REQ-CRC-009: CRC-32 ZIP . . . 16
      3.3.10. REQ-CRC-010: CRC-32 ZIP Final XOR . . . 17

3.4. Module: HASH . . . 17
3.4.1. REQ-HASH-001: DJB2 Hash Computation . . . 17
3.4.2. REQ-HASH-002: DJB2 Null Buffer Rejection . . . 18
3.4.3. REQ-HASH-003: DJB2 Result Type . . . 18
3.5. Module: HEAP . . . 19
3.5.1. REQ-HEAP-001: Heap Initialization . . . 19
3.5.2. REQ-HEAP-002: Heap Element Comparison Injection . . . 19
3.5.3. REQ-HEAP-003: Heap Insert . . . 20
3.5.4. REQ-HEAP-004: Heap Pop . . . 21
3.5.5. REQ-HEAP-005: Heap Pop Output . . . 22
3.5.6. REQ-HEAP-006: Heap Heapify . . . 22
3.5.7. REQ-HEAP-007: Heap Vtable Validation . . . 23
3.5.8. REQ-HEAP-008: Heap Dependency Validation . . . 23
3.6. Module: IO . . . 24
3.6.1. REQ-IO-001: Async IO Root Structure . . . 24
3.6.2. REQ-IO-002: Async IO Read . . . 24
3.6.3. REQ-IO-003: Async IO TryRead . . . 24
3.6.4. REQ-IO-004: Async IO ReadUntil . . . 25
3.6.5. REQ-IO-005: Async IO TryReadUntil . . . 25
3.6.6. REQ-IO-006: Async IO Write . . . 26
3.6.7. REQ-IO-007: Async IO TryWrite . . . 26
3.6.8. REQ-IO-008: Async IO Poll . . . 26
3.6.9. REQ-IO-009: I2C IO Root Structure . . . 27
3.6.10. REQ-IO-010: I2C Transfer Operation . . . 27
3.6.11. REQ-IO-011: I2C Message Type Discrimination . . . 28
3.6.12. REQ-IO-012: I2C Device Address . . . 28
3.6.13. REQ-IO-013: SPI IO Root Structure . . . 28
3.6.14. REQ-IO-014: SPI Transaction Operation . . . 29
3.7. Module: LOG . . . 29
3.7.1. REQ-LOG-001: Log Root Structure . . . 29
3.7.2. REQ-LOG-002: Debug Level Logging . . . 29
3.7.3. REQ-LOG-003: Info Level Logging . . . 30
3.7.4. REQ-LOG-004: Warning Level Logging . . . 30
3.7.5. REQ-LOG-005: Error Level Logging . . . 31
3.7.6. REQ-LOG-006: Log Initialization . . . 31
3.7.7. REQ-LOG-007: Variadic Message Format . . . 31
3.8. Module: MAP . . . 32
3.8.1. REQ-MAP-001: Map Initialization . . . 32
3.8.2. REQ-MAP-002: Map Hash Function Injection . . . 33
3.8.3. REQ-MAP-003: Map Equality Injection . . . 33
3.8.4. REQ-MAP-004: Map Set . . . 34

3.8.5. REQ-MAP-005: Map Table Full Rejection . . . 34
3.8.6. REQ-MAP-006: Map Get . . . 35
3.8.7. REQ-MAP-007: Map Key Not Found . . . 35
3.8.8. REQ-MAP-008: Map Remove . . . 36
3.8.9. REQ-MAP-009: Map Vtable Validation . . . 36
3.8.10. REQ-MAP-010: Map Dependency Validation . . . 37
3.9. Module: MATH . . . 37
3.9.1. REQ-MATH-001: Math Constants . . . 37
3.9.2. REQ-MATH-002: Vector Types . . . 38
3.9.3. REQ-MATH-003: Vector Coordinate Access . . . 38
3.9.4. REQ-MATH-004: Quaternion Type . . . 38
3.9.5. REQ-MATH-005: Vector Addition . . . 39
3.9.6. REQ-MATH-006: Vector Subtraction . . . 40
3.9.7. REQ-MATH-007: Vector Scalar Multiplication . . . 40
3.9.8. REQ-MATH-008: Vector Dot Product . . . 41
3.9.9. REQ-MATH-009: 2D Cross Product . . . 42
3.9.10. REQ-MATH-010: 3D Cross Product . . . 42
3.9.11. REQ-MATH-011: Squared L2 Norm . . . 43
3.9.12. REQ-MATH-012: Quaternion Addition . . . 44
3.9.13. REQ-MATH-013: Quaternion Subtraction . . . 44
3.9.14. REQ-MATH-014: Quaternion Scalar Multiplication . . . 44
3.9.15. REQ-MATH-015: Hamilton Product . . . 45
3.9.16. REQ-MATH-016: Quaternion Conjugate . . . 45
3.9.17. REQ-MATH-017: Quaternion Reciprocal . . . 46
3.9.18. REQ-MATH-018: Quaternion Squared Norm . . . 46
3.10. Module: MEMORY . . . 47
3.10.1. REQ-MEMORY-001: Memory Allocator Interface . . . 47
3.10.2. REQ-MEMORY-002: Allocator Pointer API Dependency . . . 47
3.10.3. REQ-MEMORY-003: Block Allocator Initialization . . . 47
3.10.4. REQ-MEMORY-004: Block Allocator Get . . . 48
3.10.5. REQ-MEMORY-005: Block Allocator Free Block Reuse . . . 49
3.10.6. REQ-MEMORY-006: Block Allocator Put . . . 49
3.10.7. REQ-MEMORY-007: Block Allocator Double Free Rejection . . . 49
3.10.8. REQ-MEMORY-008: Block Allocator Invalid Address Rejection . . . 50
3.10.9. REQ-MEMORY-009: Block Allocator Update . . . 50
3.10.10. REQ-MEMORY-010: Block Allocator No Memory Move . . . 51
3.11. Module: MODULE . . . 51
3.11.1. REQ-MODULE-001: Module Root Structure . . . 51
3.11.2. REQ-MODULE-002: Module Root Failure Handler . . . 52
3.11.3. REQ-MODULE-003: Module Derivation . . . 52
3.11.4. REQ-MODULE-004: Module Union . . . 52

3.11.5. REQ-MODULE-005: Result Type . . . 53
3.11.6. REQ-MODULE-006: Option Type . . . 53
3.12. Module: MTX . . . 54
3.12.1. REQ-MTX-001: Mutex Root Structure . . . 54
3.12.2. REQ-MTX-002: TryLock Operation . . . 54
3.12.3. REQ-MTX-003: Lock Operation . . . 54
3.12.4. REQ-MTX-004: Free Operation . . . 55
3.12.5. REQ-MTX-005: Mutex Vtable Validation . . . 55
3.13. Module: POINTER . . . 56
3.13.1. REQ-POINTER-001: Pointer Descriptor Structure . . . 56
3.13.2. REQ-POINTER-002: Pointer Copy . . . 56
3.13.3. REQ-POINTER-003: Pointer Reset . . . 57
3.13.4. REQ-POINTER-004: Pointer Descriptor Validation . . . 57
3.13.5. REQ-POINTER-005: Pointer Type Verification . . . 58
3.14. Module: QUEUE . . . 58
3.14.1. REQ-QUEUE-001: Queue Initialization . . . 58
3.14.2. REQ-QUEUE-002: Queue Initial State . . . 59
3.14.3. REQ-QUEUE-003: Queue Enqueue . . . 60
3.14.4. REQ-QUEUE-004: Queue Full Rejection . . . 60
3.14.5. REQ-QUEUE-005: Queue Dequeue . . . 61
3.14.6. REQ-QUEUE-006: Queue Empty Dequeue Rejection . . . 62
3.14.7. REQ-QUEUE-007: Queue Peek . . . 62
3.14.8. REQ-QUEUE-008: Queue Empty Peek Rejection . . . 63
3.15. Module: SB . . . 63
3.15.1. REQ-SB-001: Broker Root Structure . . . 63
3.15.2. REQ-SB-002: Broker API Interface . . . 63
3.15.3. REQ-SB-003: Broker Initialization . . . 64
3.15.4. REQ-SB-004: Pipe Derivation . . . 64
3.15.5. REQ-SB-005: Pipe Initialization . . . 65
3.15.6. REQ-SB-006: Publish Fan-Out . . . 65
3.15.7. REQ-SB-007: Publish Non-Matching Skip . . . 66
3.15.8. REQ-SB-008: Register Subscriber . . . 66
3.15.9. REQ-SB-009: Registry Full Rejection . . . 66
3.15.10. REQ-SB-010: Broker Vtable Validation . . . 67
3.15.11. REQ-SB-011: Broker Registry Validation . . . 67
3.15.12. REQ-SB-012: Pipe Validation . . . 68
3.15.13. REQ-SB-013: Single Thread Scope . . . 68
3.16. Module: SCH . . . 69
3.16.1. REQ-SCH-001: Scheduler Root Structure . . . 69
3.16.2. REQ-SCH-002: Scheduler API Interface . . . 69
3.16.3. REQ-SCH-003: Schedule Table Layout . . . 69

3.16.4. REQ-SCH-004: Execute Major Frame . . . 70
3.16.5. REQ-SCH-005: Minor Frame Period Query . . . 70
3.16.6. REQ-SCH-006: Major Frame Period Computation . . . 71
3.16.7. REQ-SCH-007: Time Source Dependency . . . 71
3.17. Module: SM . . . 72
3.17.1. REQ-SM-001: SM Root Structure . . . 72
3.17.2. REQ-SM-002: State Root Structure . . . 72
3.17.3. REQ-SM-003: State API Interface . . . 72
3.17.4. REQ-SM-004: SM Initialization . . . 73
3.17.5. REQ-SM-005: State Initialization . . . 73
3.17.6. REQ-SM-006: Get Current State . . . 74
3.17.7. REQ-SM-007: State Transition . . . 74
3.17.8. REQ-SM-008: No Transition When None . . . 75
3.17.9. REQ-SM-009: SM Verification . . . 75
3.17.10. REQ-SM-010: State Verification . . . 76
3.17.11. REQ-SM-011: Next State Pointer Verification . . . 76
3.18. Module: STACK . . . 77
3.18.1. REQ-STACK-001: Stack Initialization . . . 77
3.18.2. REQ-STACK-002: Stack Initial State . . . 78
3.18.3. REQ-STACK-003: Stack Push . . . 78
3.18.4. REQ-STACK-004: Stack Full Rejection . . . 79
3.18.5. REQ-STACK-005: Stack Pop . . . 79
3.18.6. REQ-STACK-006: Stack Empty Pop Rejection . . . 80
3.18.7. REQ-STACK-007: Stack Peek . . . 80
3.18.8. REQ-STACK-008: Stack Empty Peek Rejection . . . 81
3.19. Module: STATUS . . . 82
3.19.1. REQ-STATUS-001: Unified Status Type . . . 82
3.19.2. REQ-STATUS-002: Canonical Status Codes . . . 82
3.19.3. REQ-STATUS-003: Failure Handler Callback Type . . . 82
3.19.4. REQ-STATUS-004: Failure Handler Non-Interference . . . 83
3.20. Module: SYS . . . 83
3.20.1. REQ-SYS-001: Zero Dynamic Allocation . . . 83
3.20.2. REQ-SYS-002: Caller-Owned Memory . . . 84
3.20.3. REQ-SYS-003: Freestanding C11 Compatibility . . . 84
3.20.4. REQ-SYS-004: Explicit Status Code Returns . . . 85
3.20.5. REQ-SYS-005: No Silent Failures . . . 85
3.20.6. REQ-SYS-006: Dependency Injection via Vtables . . . 86
3.20.7. REQ-SYS-007: No Global Dependencies . . . 86
3.20.8. REQ-SYS-008: Precondition Verification . . . 86
3.20.9. REQ-SYS-009: No Global Mutable State . . . 87
3.20.10. REQ-SYS-010: API Vtable Validation . . . 87

3.20.11. REQ-SYS-011: Injected Dependency Validation . . . 88
3.21. Module: TIME . . . 89
3.21.1. REQ-TIME-001: Timestamp Representation . . . 89
3.21.2. REQ-TIME-002: Time API Interface . . . 89
3.21.3. REQ-TIME-003: AddTime Operation . . . 89
3.21.4. REQ-TIME-004: AddTime Subsecond Carry-Over . . . 90
3.21.5. REQ-TIME-005: SubtractTime Operation . . . 90
3.21.6. REQ-TIME-006: SubtractTime Subsecond Borrow . . . 91
3.21.7. REQ-TIME-007: SubtractTime Negative Saturation . . . 91
3.21.8. REQ-TIME-008: SubtractTime Negative Error . . . 92
3.21.9. REQ-TIME-009: Timestamp to Nanoseconds Conversion . . . 92
3.21.10. REQ-TIME-010: Timestamp to Microseconds Conversion . . . 93
3.21.11. REQ-TIME-011: Timestamp to Milliseconds Conversion . . . 93
3.21.12. REQ-TIME-012: Nanoseconds to Timestamp Conversion . . . 94
3.21.13. REQ-TIME-013: Microseconds to Timestamp Conversion . . . 94
3.21.14. REQ-TIME-014: Milliseconds to Timestamp Conversion . . . 95
3.21.15. REQ-TIME-015: Timestamp to Double Conversion . . . 95
3.21.16. REQ-TIME-016: Double to Timestamp Conversion . . . 95
3.21.17. REQ-TIME-017: Double to Timestamp Range Rejection . . . 96
3.21.18. REQ-TIME-018: Conversion Overflow Detection . . . 96
3.21.19. REQ-TIME-019: Null Pointer Rejection . . . 97
4. Traceability Validation Summary . . . 98

1. Introduction

1.1. Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for LibJuno, a freestanding C11 embedded systems micro-framework.

This document follows the IEEE 830-1998 standard structure.

1.2. Scope

LibJuno provides data structures, memory management primitives, CRC algorithms, and other utilities for embedded systems development. It requires zero dynamic memory allocation and supports freestanding C11 environments.

1.3. Definitions and Acronyms

Term	Definition
Freestanding	A C implementation that does not require a hosted standard library
DI	Dependency Injection — a pattern where dependencies are provided to a component rather than created internally
Vtable	A struct of function pointers enabling polymorphic dispatch
SRS	Software Requirements Specification
RTM	Requirements Traceability Matrix

1.4. References

- IEEE 830-1998: Recommended Practice for Software Requirements Specifications
- LibJuno README and documentation

2. Overall Description

2.1. Product Perspective

LibJuno is a standalone C11 library designed to be used as a dependency in embedded systems projects. It makes no assumptions about the target platform and uses dependency injection to remain adaptable.

2.2. Design Constraints

- No dynamic memory allocation (`malloc`, `calloc`, `realloc`, `free`)
- All memory is caller-owned and injected
- Must compile as freestanding C11 (`-nostdlib -ffreestanding`)
- All warnings treated as errors (`-Werror`)

2.3. Assumptions and Dependencies

- A C11-compliant compiler is available
- CMake ≥ 3.10 is available for building
- Unity test framework is available for testing (build-time only)

3. Specific Requirements

3.1. Module: APP

3.1.1. REQ-APP-001: Application Root Structure

Description

The application root shall embed a module root with an API vtable pointer.

Rationale

The application root provides the standard vtable-based interface that the scheduler and other framework components use to invoke application lifecycle hooks polymorphically.

Verification Method

Inspection

Uses (parent)

REQ-SYS-001, REQ-SYS-002, REQ-SYS-003, REQ-SYS-006, REQ-SYS-007, REQ-SYS-009, REQ-SYS-010

Code Traces

- `include/juno/app/app_api.h:52` (JUNO_MODULE_ROOT)

3.1.2. REQ-APP-002: OnStart Lifecycle Hook

Description

The application API vtable shall define an OnStart operation called once to initialize resources.

Rationale

OnStart provides a one-time initialization point for applications to set up resources before the scheduler begins periodic execution.

Verification Method

Inspection

Uses (parent)

REQ-APP-001

Code Traces

- `include/juno/app/app_api.h:58` (JUNO_STATUS_T)

3.1.3. REQ-APP-003: OnProcess Lifecycle Hook

Description

The application API vtable shall define an OnProcess operation that may be invoked repeatedly.

Rationale

OnProcess is the primary execution hook invoked by the scheduler on each minor frame, providing the periodic processing entry point for application logic.

Verification Method

Inspection

Uses (parent)

REQ-APP-001

Code Traces

- `include/juno/app/app_api.h:61` (JUNO_STATUS_T)

3.1.4. REQ-APP-004: OnExit Lifecycle Hook

Description

The application API vtable shall define an OnExit operation called once prior to shutdown.

Rationale

OnExit provides a cleanup point for applications to release or reset resources before shutdown, ensuring orderly termination.

Verification Method

Inspection

Uses (parent)

REQ-APP-001

Code Traces

- `include/juno/app/app_api.h:64` (JUNO_STATUS_T)

3.2. Module: ARRAY

3.2.1. REQ-ARRAY-001: Array Initialization

Description

The array module shall initialize from a caller-provided API vtable and capacity.

Rationale

The array is the foundational data structure upon which queue, stack, heap, and map are built; it must initialize from caller-provided storage to satisfy the zero-allocation constraint.

Verification Method

Test

Uses (parent)

REQ-SYS-001, REQ-SYS-002, REQ-SYS-003, REQ-SYS-006, REQ-SYS-007, REQ-SYS-009

Code Traces

- `include/juno/ds/array_api.h:163` (JunoDs_ArrayInit)

Test Traces

- `tests/test_array.c:150` (test_array_init_success)
- `tests/test_array.c:162` (test_array_init_null_array)

3.2.2. REQ-ARRAY-002: Array SetAt

Description

The array shall copy a value into a slot at a given index in O(1) time.

Rationale

SetAt is the fundamental write operation, using the pointer API to copy values into slots in a type-safe manner.

Verification Method

Test

Uses (parent)

REQ-ARRAY-001

Code Traces

- `include/juno/ds/array_api.h:82` (JUNO_STATUS_T)

Test Traces

- `tests/test_array.c:174` (test_array_set_at)

3.2.3. REQ-ARRAY-003: Array GetAt

Description

The array shall return a pointer descriptor to the slot at a given index in O(1) time.

Rationale

GetAt returns a pointer descriptor rather than a raw pointer, maintaining the type-safe access model throughout the data structure hierarchy.

Verification Method

Test

Uses (parent)

REQ-ARRAY-001

Code Traces

- `include/juno/ds/array_api.h:89` (JUNO_RESULT_POINTER_T)

Test Traces

- `tests/test_array.c:184` (test_array_get_at)

3.2.4. REQ-ARRAY-004: Array RemoveAt

Description

The array shall clear the slot at a given index in O(1) time.

Rationale

Clearing a slot via the pointer Reset operation ensures no stale data remains, preventing

downstream consumers from reading invalid values.

Verification Method

Test

Uses (parent)

REQ-ARRAY-001

Code Traces

- `include/juno/ds/array_api.h:96` (JUNO_STATUS_T)

Test Traces

- `tests/test_array.c:195` (test_array_remove_at)

3.2.5. REQ-ARRAY-005: Array Bounds Checking

Description

The array shall return JUNO_STATUS_OOB_ERROR when an index is outside its capacity.

Rationale

Explicit bounds checking prevents out-of-bounds memory access, which would cause silent corruption or undefined behavior on bare-metal targets lacking an MMU.

Verification Method

Test

Uses (parent)

REQ-SYS-008

Code Traces

- `include/juno/ds/array_api.h:140` (JunoDs_ArrayVerifyIndex)

Test Traces

- `tests/test_array.c:208` (test_array_set_at_oob)
- `tests/test_array.c:218` (test_array_get_at_oob)
- `tests/test_array.c:226` (test_array_remove_at_oob)
- `tests/test_queue.c:913` (test_queue_array_api_get_at_out_of_bounds)
- `tests/test_queue.c:923` (test_queue_array_api_remove_at_out_of_bounds)
- `tests/test_stack.c:926` (test_stack_array_api_get_at_out_of_bounds)
- `tests/test_stack.c:936` (test_stack_array_api_remove_at_out_of_bounds)

3.2.6. REQ-ARRAY-006: Array Vtable Validation

Description

The array module shall verify that the array API vtable entries are non-null.

Rationale

Validates the vtable at runtime to ensure memory safety before dispatching any operation through function pointers.

Verification Method

Test

Uses (parent)

REQ-SYS-010

Code Traces

- `include/juno/ds/array_api.h:120` (JunoDs_ArrayVerify)

Test Traces

- `tests/test_array.c:238` (test_array_verify_null_api)
- `tests/test_queue.c:892` (test_queue_array_api_set_at_invalid_api)
- `tests/test_stack.c:905` (test_stack_array_api_set_at_invalid_api)
- `tests/test_stack.c:946` (test_stack_array_api_null_array)

3.2.7. REQ-ARRAY-007: Array Capacity Validation

Description

The array module shall verify that the array capacity is non-zero.

Rationale

A zero-capacity array has no backing storage and cannot perform any useful operation; rejecting it early prevents division-by-zero or empty-loop errors in dependent modules.

Verification Method

Test

Uses (parent)

REQ-SYS-008

Code Traces

- `include/juno/ds/array_api.h:120` (JunoDs_ArrayVerify)
- `include/juno/ds/array_api.h:163` (JunoDs_ArrayInit)

Test Traces

- `tests/test_array.c:253` (test_array_verify_zero_capacity)
- `tests/test_array.c:264` (test_array_init_zero_capacity)

3.3. Module: CRC

3.3.1. REQ-CRC-001: CRC Streaming Update

Description

Each CRC function shall accept a running CRC value, data buffer, and size, returning the updated CRC value.

Rationale

Streaming CRC computation allows data integrity checks over arbitrarily large or fragmented data without requiring the entire payload to be buffered in memory.

Verification Method

Test

Uses (parent)

REQ-SYS-001, REQ-SYS-003, REQ-SYS-009

Code Traces

- `src/juno_arc.c:21` (Juno_CrcArcUpdate)
- `src/juno_binhex.c:21` (Juno_CrcBinhexUpdate)
- `src/juno_ccitt.c:21` (Juno_CrcCcittUpdate)
- `src/juno_ccitt32.c:20` (Juno_CrcCcitt32Update)
- `src/juno_kermit.c:21` (Juno_CrcKermitUpdate)
- `src/juno_zip.c:20` (Juno_CrcZipUpdate)

Test Traces

- `tests/test_crc.c:81` (test_various_crc_behaviour)

3.3.2. REQ-CRC-002: CRC Null Safety

Description

Each CRC function shall return 0 when passed a NULL data pointer.

Rationale

Returning a defined value for NULL input ensures the function is safe to call unconditionally, aligning with the no-silent-failures constraint by producing a predictable result rather than crashing.

Verification Method

Test

Uses (parent)

REQ-SYS-005

Code Traces

- `src/juno_arc.c:21` (Juno_CrcArcUpdate)
- `src/juno_binhex.c:21` (Juno_CrcBinhexUpdate)

- `src/juno_ccitt.c:21` (Juno_CrcCcittUpdate)
- `src/juno_ccitt32.c:20` (Juno_CrcCcitt32Update)
- `src/juno_kermit.c:21` (Juno_CrcKermitUpdate)
- `src/juno_zip.c:20` (Juno_CrcZipUpdate)

Test Traces

- `tests/test_crc.c:70` (test_empty_crc_all)

3.3.3. REQ-CRC-003: CRC Zero Length Safety

Description

Each CRC function shall return 0 when passed a zero data size.

Rationale

Returning a defined value for zero-length input ensures the function is safe to call on empty buffers without special-casing at the call site.

Verification Method

Test

Uses (parent)

REQ-SYS-005

Code Traces

- `src/juno_arc.c:21` (Juno_CrcArcUpdate)
- `src/juno_binhex.c:21` (Juno_CrcBinhexUpdate)
- `src/juno_ccitt.c:21` (Juno_CrcCcittUpdate)
- `src/juno_ccitt32.c:20` (Juno_CrcCcitt32Update)
- `src/juno_kermit.c:21` (Juno_CrcKermitUpdate)
- `src/juno_zip.c:20` (Juno_CrcZipUpdate)

Test Traces

- `tests/test_crc.c:70` (test_empty_crc_all)

3.3.4. REQ-CRC-004: CRC-16 ARC

Description

The CRC module shall compute a CRC-16/ARC update with initial seed 0.

Rationale

CRC-16/ARC is a widely used 16-bit CRC variant in legacy serial protocols and embedded communication stacks.

Verification Method

Test

Uses (parent)

REQ-CRC-001

Code Traces

- `src/juno_arc.c:21` (Juno_CrcArcUpdate)

Test Traces

- `tests/test_crc.c:81` (test_various_crc_behaviour)

3.3.5. REQ-CRC-005: CRC-16 BinHex

Description

The CRC module shall compute a BinHex 16-bit CRC update with initial seed 0.

Rationale

BinHex CRC is used in Macintosh file encoding and certain archive formats that embedded systems may need to interoperate with.

Verification Method

Test

Uses (parent)

REQ-CRC-001

Code Traces

- `src/juno_binhex.c:21` (Juno_CrcBinhexUpdate)

Test Traces

- `tests/test_crc.c:81` (test_various_crc_behaviour)

3.3.6. REQ-CRC-006: CRC-16 CCITT

Description

The CRC module shall compute a CRC-16/CCITT update with initial seed 0xFFFF.

Rationale

CRC-16/CCITT is a standard integrity check in telecommunications (X.25, HDLC) and is commonly required for embedded communication interfaces.

Verification Method

Test

Uses (parent)

REQ-CRC-001

Code Traces

- `src/juno_ccitt.c:21` (Juno_CrcCcittUpdate)

Test Traces

- `tests/test_crc.c:56` (test_ccitt_crc)

3.3.7. REQ-CRC-007: CRC-16 Kermit

Description

The CRC module shall compute a CRC-16/Kermit update with initial seed 0.

Rationale

CRC-16/Kermit is used in the Kermit file transfer protocol and certain embedded serial communication stacks.

Verification Method

Test

Uses (parent)

REQ-CRC-001

Code Traces

- `src/juno_kermit.c:21` (Juno_CrcKermitUpdate)

Test Traces

- `tests/test_crc.c:81` (test_various_crc_behaviour)

3.3.8. REQ-CRC-008: CRC-32 CCITT32

Description

The CRC module shall compute a 32-bit CCITT CRC update with initial seed 0xFFFFFFFF.

Rationale

A 32-bit CCITT CRC provides stronger error detection than 16-bit variants and is needed for larger data payloads and storage integrity checks.

Verification Method

Test

Uses (parent)

REQ-CRC-001

Code Traces

- `src/juno_ccitt32.c:20` (Juno_CrcCcitt32Update)

Test Traces

- `tests/test_crc.c:81` (test_various_crc_behaviour)

3.3.9. REQ-CRC-009: CRC-32 ZIP

Description

The CRC module shall compute a CRC-32/ZIP update with initial seed 0xFFFFFFFF.

Rationale

CRC-32/ZIP is the de facto standard for file and stream integrity (ZIP, PNG, Ethernet FCS) and is commonly required for interoperability.

Verification Method

Test

Uses (parent)

REQ-CRC-001

Code Traces

- `src/juno_zip.c:20` (Juno_CrcZipUpdate)

Test Traces

- `tests/test_crc.c:37` (test_zip_crc)

3.3.10. REQ-CRC-010: CRC-32 ZIP Final XOR

Description

The CRC-32/ZIP function shall apply a final XOR of 0xFFFFFFFF to the computed value when data is processed.

Rationale

The final XOR is part of the CRC-32/ZIP specification and is required for the computed value to match reference implementations and standard test vectors.

Verification Method

Test

Uses (parent)

REQ-CRC-009

Code Traces

- `src/juno_zip.c:20` (Juno_CrcZipUpdate)

Test Traces

- `tests/test_crc.c:37` (test_zip_crc)

3.4. Module: HASH

3.4.1. REQ-HASH-001: DJB2 Hash Computation

Description

The hash module shall compute the djb2 hash over a byte buffer with initial seed 5381.

Rationale

DJB2 is a simple, fast hash function suitable for hash map key distribution in resource-constrained environments where cryptographic strength is not required.

Verification Method

Test

Uses (parent)

REQ-SYS-001, REQ-SYS-003, REQ-SYS-004, REQ-SYS-005, REQ-SYS-008, REQ-SYS-009

Code Traces

- `include/juno/hash/hash_djb2.h:47` (JunoHash_Djb2)

Test Traces

- `tests/test_hash.c:37` (test_hash_djb2_known_value)
- `tests/test_hash.c:52` (test_hash_djb2_empty_buffer)
- `tests/test_hash.c:62` (test_hash_djb2_different_inputs_differ)

3.4.2. REQ-HASH-002: DJB2 Null Buffer Rejection

Description

The hash module shall return JUNO_STATUS_NULLPTR_ERROR when passed a NULL data pointer.

Rationale

Rejecting NULL input prevents undefined behavior from dereferencing an invalid buffer pointer, ensuring runtime memory safety.

Verification Method

Test

Uses (parent)

REQ-SYS-008

Code Traces

- `include/juno/hash/hash_djb2.h:47` (JunoHash_Djb2)

Test Traces

- `tests/test_hash.c:78` (test_hash_djb2_null_buffer)

3.4.3. REQ-HASH-003: DJB2 Result Type

Description

The hash module shall return the hash value in a result type combining a status code with a size_t payload.

Rationale

Returning the hash in a result type couples the computed value with a status code, enabling

callers to distinguish between a valid hash and an error without ambiguity.

Verification Method

Inspection

Uses (parent)

REQ-SYS-004

Code Traces

- `include/juno/hash/hash_djb2.h:47` (JunoHash_Djb2)

3.5. Module: HEAP

3.5.1. REQ-HEAP-001: Heap Initialization

Description

The heap module shall initialize from a caller-provided backing array.

Rationale

A priority queue is essential for scheduling and event-driven systems; initializing from a caller-provided array satisfies the zero-allocation constraint.

Verification Method

Test

Uses (parent)

REQ-SYS-001, REQ-SYS-002, REQ-SYS-003, REQ-SYS-004, REQ-SYS-005, REQ-SYS-006, REQ-SYS-007, REQ-SYS-008, REQ-SYS-009, REQ-ARRAY-001

Code Traces

- `src/juno_heap.c:32` (JunoDs_Heap_Init)

Test Traces

- `tests/test_heap.c:353` (test_heap_init_nominal)
- `tests/test_heap.c:363` (test_heap_init_null_heap)
- `tests/test_heap.c:373` (test_heap_init_null_api)
- `tests/test_heap.c:383` (test_heap_init_zero_capacity)

3.5.2. REQ-HEAP-002: Heap Element Comparison Injection

Description

The heap module shall accept an injected element pointer API with Compare and Swap operations.

Rationale

Injecting Compare and Swap operations allows the heap to be generic over element type, supporting min-heaps, max-heaps, and custom orderings without code duplication.

Verification Method

Test

Uses (parent)

REQ-SYS-007, REQ-HEAP-001

Code Traces

- `src/juno_heap.c:32` (JunoDs_Heap_Init)

Test Traces

- `tests/test_heap.c:526` (test_heap_min_heap_insert_and_property)
- `tests/test_heap.c:709` (test_heap_update_with_compare_error)
- `tests/test_heap.c:727` (test_heap_update_with_swap_error)
- `tests/test_heap.c:825` (test_heap_siftdown_with_compare_error)
- `tests/test_heap.c:844` (test_heap_siftdown_with_swap_error)
- `tests/test_heap.c:909` (test_heap_compare_function_error)
- `tests/test_heap.c:931` (test_heap_swap_function_error)
- `tests/test_heap.c:994` (test_heap_mixed_min_max_operations)

3.5.3. REQ-HEAP-003: Heap Insert

Description

The heap shall insert an element and restore the heap property in O(log n) time.

Rationale

O(log n) insertion maintains the heap property efficiently, providing predictable worst-case timing for real-time priority queue operations.

Verification Method

Test

Uses (parent)

REQ-HEAP-001

Code Traces

- `src/juno_heap.c:258` (JunoDs_Heap_Insert)

Test Traces

- `tests/test_heap.c:415` (test_heap_insert_single_item_max_heap)
- `tests/test_heap.c:432` (test_heap_insert_multiple_items_max_heap)
- `tests/test_heap.c:453` (test_heap_insert_to_full_capacity)
- `tests/test_heap.c:472` (test_heap_insert_overflow)
- `tests/test_heap.c:495` (test_heap_insert_null_heap)
- `tests/test_heap.c:505` (test_heap_insert_invalid_pointer)

- `tests/test_heap.c:526` (test_heap_min_heap_insert_and_property)
- `tests/test_heap.c:653` (test_heap_update_single_element)
- `tests/test_heap.c:669` (test_heap_update_bubble_up)
- `tests/test_heap.c:691` (test_heap_update_empty_heap)
- `tests/test_heap.c:702` (test_heap_update_null_heap)
- `tests/test_heap.c:957` (test_heap_stress_insert_pop_cycles)
- `tests/test_heap.c:994` (test_heap_mixed_min_max_operations)
- `tests/test_heap.c:1033` (test_heap_single_element_capacity)
- `tests/test_heap.c:1058` (test_heap_data_integrity_after_operations)

3.5.4. REQ-HEAP-004: Heap Pop

Description

The heap shall remove the root element and restore the heap property in O(log n) time.

Rationale

O(log n) removal of the root element enables efficient extraction of the highest-priority item, which is the primary operation for priority-queue consumers.

Verification Method

Test

Uses (parent)

REQ-HEAP-001

Code Traces

- `src/juno_heap.c:325` (JunoDs_Heap_Pop)

Test Traces

- `tests/test_heap.c:550` (test_heap_pop_single_item_max_heap)
- `tests/test_heap.c:574` (test_heap_pop_multiple_items_max_heap)
- `tests/test_heap.c:610` (test_heap_pop_empty_heap)
- `tests/test_heap.c:622` (test_heap_pop_null_heap)
- `tests/test_heap.c:631` (test_heap_pop_invalid_pointer)
- `tests/test_heap.c:749` (test_heap_siftdown_from_root)
- `tests/test_heap.c:767` (test_heap_siftdown_from_middle)
- `tests/test_heap.c:787` (test_heap_siftdown_leaf_node)
- `tests/test_heap.c:807` (test_heap_siftdown_empty_heap)
- `tests/test_heap.c:818` (test_heap_siftdown_null_heap)
- `tests/test_heap.c:957` (test_heap_stress_insert_pop_cycles)
- `tests/test_heap.c:994` (test_heap_mixed_min_max_operations)

- `tests/test_heap.c:1033` (test_heap_single_element_capacity)
- `tests/test_heap.c:1058` (test_heap_data_integrity_after_operations)

3.5.5. REQ-HEAP-005: Heap Pop Output

Description

The heap Pop operation shall copy the removed root element into a caller-provided pointer descriptor.

Rationale

Copying the removed element into a caller-provided descriptor ensures the caller receives the value without exposing internal array storage or requiring dynamic allocation for the output.

Verification Method

Test

Uses (parent)

REQ-HEAP-004

Code Traces

- `src/juno_heap.c:325` (JunoDs_Heap_Pop)

Test Traces

- `tests/test_heap.c:550` (test_heap_pop_single_item_max_heap)
- `tests/test_heap.c:574` (test_heap_pop_multiple_items_max_heap)
- `tests/test_heap.c:1058` (test_heap_data_integrity_after_operations)

3.5.6. REQ-HEAP-006: Heap Heapify

Description

The heap shall transform the underlying array into a valid heap in O(n) time.

Rationale

O(n) heapify enables efficient bulk construction of a heap from an existing array, which is asymptotically faster than n individual insertions.

Verification Method

Test

Uses (parent)

REQ-HEAP-001

Code Traces

- `src/juno_heap.c:284` (JunoDs_Heap_Heapify)

Test Traces

- `tests/test_heap.c:867` (test_heap_heapify_from_unsorted_array)

- `tests/test_heap.c:886` (test_heap_heapify_empty_heap)
- `tests/test_heap.c:898` (test_heap_heapify_null_heap)

3.5.7. REQ-HEAP-007: Heap Vtable Validation

Description

The heap module shall verify that the heap API vtable entries are non-null before each operation.

Rationale

Validates the vtable at runtime to ensure memory safety before dispatching any heap operation through function pointers.

Verification Method

Test

Uses (parent)

REQ-SYS-010

Code Traces

- `include/juno/ds/heap_api.h:227` (JunoDs_Heap_Verify)

Test Traces

- `tests/test_heap.c:397` (test_heap_verify_null_heap)
- `tests/test_heap.c:403` (test_heap_verify_invalid_api)

3.5.8. REQ-HEAP-008: Heap Dependency Validation

Description

The heap module shall verify that the backing array and heap pointer API dependencies are valid before each operation.

Rationale

Validates the injected backing array and comparison API at runtime to catch misconfigured or partially initialized heap instances before they cause undefined behavior.

Verification Method

Test

Uses (parent)

REQ-SYS-011

Code Traces

- `include/juno/ds/heap_api.h:227` (JunoDs_Heap_Verify)

Test Traces

- `tests/test_heap.c:373` (test_heap_init_null_api)
- `tests/test_heap.c:383` (test_heap_init_zero_capacity)

3.6. Module: IO

3.6.1. REQ-IO-001: Async IO Root Structure

Description

The async IO root shall embed a module root with an API vtable pointer.

Rationale

An abstract async IO interface decouples application-level communication logic from platform-specific UART, socket, or DMA implementations, enabling portability and testability.

Verification Method

Inspection

Uses (parent)

REQ-SYS-001, REQ-SYS-002, REQ-SYS-003, REQ-SYS-006, REQ-SYS-007, REQ-SYS-009, REQ-SYS-010

Code Traces

- `include/juno/io/async_io_api.h:47` (JUNO_MODULE_ROOT)

3.6.2. REQ-IO-002: Async IO Read

Description

The async IO API vtable shall define a Read operation that reads up to the requested number of bytes and updates the size with bytes actually read.

Rationale

Read with actual-bytes-read feedback allows callers to handle partial reads, which is the norm for non-blocking and DMA-based IO on embedded targets.

Verification Method

Inspection

Uses (parent)

REQ-IO-001

Code Traces

- `include/juno/io/async_io_api.h:53` (JUNO_STATUS_T)

3.6.3. REQ-IO-003: Async IO TryRead

Description

The async IO API vtable shall define a TryRead operation that reads with a timeout in microseconds.

Rationale

TryRead with a timeout enables bounded-time read attempts, which is essential for real-time

systems that cannot block indefinitely on IO.

Verification Method

Inspection

Uses (parent)

REQ-IO-001

Code Traces

- `include/juno/io/async_io_api.h:56` (JUNO_STATUS_T)

3.6.4. REQ-IO-004: Async IO ReadUntil

Description

The async IO API vtable shall define a ReadUntil operation that reads until any stop character is received.

Rationale

ReadUntil enables delimiter-based framing (e.g., newline-terminated protocols) without requiring the caller to implement byte-by-byte polling.

Verification Method

Inspection

Uses (parent)

REQ-IO-001

Code Traces

- `include/juno/io/async_io_api.h:59` (JUNO_STATUS_T)

3.6.5. REQ-IO-005: Async IO TryReadUntil

Description

The async IO API vtable shall define a TryReadUntil operation that reads until a stop character or timeout.

Rationale

TryReadUntil combines delimiter-based framing with a timeout, preventing indefinite blocking when the delimiter never arrives.

Verification Method

Inspection

Uses (parent)

REQ-IO-001

Code Traces

- `include/juno/io/async_io_api.h:62` (JUNO_STATUS_T)

3.6.6. REQ-IO-006: Async IO Write

Description

The async IO API vtable shall define a Write operation that writes up to the requested number of bytes and updates the size with bytes actually written.

Rationale

Write with actual-bytes-written feedback allows callers to handle partial writes, which is common with buffered or DMA-based IO on embedded targets.

Verification Method

Inspection

Uses (parent)

REQ-IO-001

Code Traces

- `include/juno/io/async_io_api.h:65` (JUNO_STATUS_T)

3.6.7. REQ-IO-007: Async IO TryWrite

Description

The async IO API vtable shall define a TryWrite operation that writes with a timeout in microseconds.

Rationale

TryWrite with a timeout enables bounded-time write attempts, preventing real-time tasks from stalling on a full output buffer.

Verification Method

Inspection

Uses (parent)

REQ-IO-001

Code Traces

- `include/juno/io/async_io_api.h:68` (JUNO_STATUS_T)

3.6.8. REQ-IO-008: Async IO Poll

Description

The async IO API vtable shall define a Poll operation that checks for input readiness within a timeout.

Rationale

Poll enables non-blocking readiness checks, allowing applications to multiplex across IO channels or perform other work while waiting for input.

Verification Method

Inspection

Uses (parent)

REQ-IO-001

Code Traces

- `include/juno/io/async_io_api.h:71` (JUNO_STATUS_T)

3.6.9. REQ-IO-009: I2C IO Root Structure

Description

The I2C IO root shall embed a module root with an API vtable pointer.

Rationale

An abstract I2C interface decouples sensor and peripheral drivers from platform-specific I2C hardware, enabling portability and testability across different MCU families.

Verification Method

Inspection

Uses (parent)

REQ-SYS-006

Code Traces

- `include/juno/io/i2c_io_api.h:108` (JUNO_MODULE_ROOT)

3.6.10. REQ-IO-010: I2C Transfer Operation

Description

The I2C IO API vtable shall define a Transfer operation that processes an array of read/write messages.

Rationale

A multi-message transfer operation enables compound I2C transactions (e.g., write-then-read) in a single call, which is the standard pattern for register-based peripherals.

Verification Method

Inspection

Uses (parent)

REQ-IO-009

Code Traces

- `include/juno/io/i2c_io_api.h:134` (JUNO_STATUS_T)

3.6.11. REQ-IO-011: I2C Message Type Discrimination

Description

Each I2C message shall contain a header with a type field distinguishing read from write messages.

Rationale

A type field in the message header enables the transfer operation to dispatch read vs. write behavior without requiring separate API calls for each direction.

Verification Method

Inspection

Uses (parent)

REQ-IO-010

Code Traces

- `include/juno/io/i2c_io_api.h:58` (<unknown>)

3.6.12. REQ-IO-012: I2C Device Address

Description

Each I2C message header shall contain the target device address.

Rationale

Including the device address in each message header allows a single transfer call to address multiple devices on the same bus, supporting multi-device transactions.

Verification Method

Inspection

Uses (parent)

REQ-IO-011

Code Traces

- `include/juno/io/i2c_io_api.h:58` (<unknown>)

3.6.13. REQ-IO-013: SPI IO Root Structure

Description

The SPI IO root shall embed a module root with an API vtable pointer.

Rationale

An abstract SPI interface decouples sensor and peripheral drivers from platform-specific SPI hardware, enabling portability and testability across different MCU families.

Verification Method

Inspection

Uses (parent)

REQ-SYS-006

Code Traces

- `include/juno/io/spi_io_api.h:46` (JUNO_MODULE_ROOT)

3.6.14. REQ-IO-014: SPI Transaction Operation

Description

The SPI IO API vtable shall define a Transaction operation for full-duplex read/write.

Rationale

Full-duplex transaction is the native SPI operation model; providing it as a single call allows the platform backend to optimize DMA or hardware-level simultaneous read/write.

Verification Method

Inspection

Uses (parent)

REQ-IO-013

Code Traces

- `include/juno/io/spi_io_api.h:58` (JUNO_STATUS_T)

3.7. Module: LOG

3.7.1. REQ-LOG-001: Log Root Structure

Description

The log root shall embed a module root with an API vtable pointer.

Rationale

An abstract log interface decouples application diagnostic output from platform-specific logging backends (UART, semihosting, telemetry), enabling portability and testability.

Verification Method

Inspection

Uses (parent)

REQ-SYS-001, REQ-SYS-003, REQ-SYS-006, REQ-SYS-009, REQ-SYS-010

Code Traces

- `include/juno/log/log_api.h:51` (JUNO_MODULE_ROOT)

3.7.2. REQ-LOG-002: Debug Level Logging

Description

The log API vtable shall define a LogDebug operation for debug-level messages.

Rationale

Debug-level messages provide verbose diagnostic information useful during development and integration testing.

Verification Method

Inspection

Uses (parent)

REQ-LOG-001

Code Traces

- `include/juno/log/log_api.h:57` (JUNO_STATUS_T)

3.7.3. REQ-LOG-003: Info Level Logging

Description

The log API vtable shall define a LogInfo operation for info-level messages.

Rationale

Info-level messages communicate normal operational status and milestones, useful for runtime monitoring.

Verification Method

Inspection

Uses (parent)

REQ-LOG-001

Code Traces

- `include/juno/log/log_api.h:60` (JUNO_STATUS_T)

3.7.4. REQ-LOG-004: Warning Level Logging

Description

The log API vtable shall define a LogWarning operation for warning-level messages.

Rationale

Warning-level messages flag recoverable anomalies that may require operator attention but do not halt execution.

Verification Method

Inspection

Uses (parent)

REQ-LOG-001

Code Traces

- `include/juno/log/log_api.h:63` (JUNO_STATUS_T)

3.7.5. REQ-LOG-005: Error Level Logging

Description

The log API vtable shall define a LogError operation for error-level messages.

Rationale

Error-level messages flag critical failures that may require corrective action or system recovery.

Verification Method

Inspection

Uses (parent)

REQ-LOG-001

Code Traces

- `include/juno/log/log_api.h:66` (JUNO_STATUS_T)

3.7.6. REQ-LOG-006: Log Initialization

Description

The log module shall initialize with an injected API vtable and failure handler.

Rationale

Initialization with an injected vtable and failure handler wires the log module to a platform-specific backend while maintaining the dependency injection pattern.

Verification Method

Test

Uses (parent)

REQ-SYS-002, REQ-SYS-007, REQ-LOG-001

Code Traces

- `include/juno/log/log_api.h:71` (JunoLog_LogInit)

Test Traces

- `tests/test_log.c:94` (test_log_init_success)
- `tests/test_log.c:102` (test_log_init_null_root)

3.7.7. REQ-LOG-007: Variadic Message Format

Description

Each log operation shall accept a printf-like format string with variadic arguments.

Rationale

Printf-like variadic formatting is the standard embedded logging convention, allowing callers to compose messages inline without pre-formatting into a separate buffer.

Verification Method

Inspection

Uses (parent)

REQ-LOG-001

Code Traces

- `include/juno/log/log_api.h:57` (JUNO_STATUS_T)
- `include/juno/log/log_api.h:60` (JUNO_STATUS_T)
- `include/juno/log/log_api.h:63` (JUNO_STATUS_T)
- `include/juno/log/log_api.h:66` (JUNO_STATUS_T)

3.8. Module: MAP

3.8.1. REQ-MAP-001: Map Initialization

Description

The map module shall initialize from a caller-provided backing array.

Rationale

A hash map provides O(1) average-case key-value lookup, which is essential for configuration registries, symbol tables, and message routing in embedded systems; initializing from a caller-provided array satisfies the zero-allocation constraint.

Verification Method

Test

Uses (parent)

REQ-SYS-001, REQ-SYS-002, REQ-SYS-003, REQ-SYS-004, REQ-SYS-005, REQ-SYS-006, REQ-SYS-007, REQ-SYS-008, REQ-SYS-009, REQ-ARRAY-001

Code Traces

- `src/juno_map.c:34` (JunoDs_MapInit)

Test Traces

- `tests/test_map.c:322` (test_map_init_nominal)
- `tests/test_map.c:336` (test_map_init_null_map)
- `tests/test_map.c:343` (test_map_init_null_api)
- `tests/test_map.c:351` (test_map_init_zero_capacity)

3.8.2. REQ-MAP-002: Map Hash Function Injection

Description

The map module shall accept an injected hashable pointer API with Hash and IsValueNull operations.

Rationale

Injecting the hash function decouples the map from a specific hash algorithm, allowing integrators to choose an algorithm suited to their key type and collision characteristics.

Verification Method

Test

Uses (parent)

REQ-SYS-007, REQ-MAP-001

Code Traces

- `src/juno_map.c:34` (JunoDs_MapInit)

Test Traces

- `tests/test_map.c:322` (test_map_init_nominal)
- `tests/test_map.c:901` (test_map_entry_hash_nominal)
- `tests/test_map.c:912` (test_map_entry_is_null_true)
- `tests/test_map.c:923` (test_map_entry_is_null_false)

3.8.3. REQ-MAP-003: Map Equality Injection

Description

The map module shall accept an injected value pointer API with an Equals operation for key comparison.

Rationale

Injecting the equality operation decouples the map from a specific key type, allowing arbitrary key comparisons without hard-coded logic.

Verification Method

Test

Uses (parent)

REQ-SYS-007, REQ-MAP-001

Code Traces

- `src/juno_map.c:34` (JunoDs_MapInit)

Test Traces

- `tests/test_map.c:322` (test_map_init_nominal)
- `tests/test_map.c:873` (test_map_entry_equals_same_key)

- `tests/test_map.c:887` (test_map_entry_equals_different_key)

3.8.4. REQ-MAP-004: Map Set

Description

The map shall insert or update a key-value pair using linear probing.

Rationale

Linear probing provides a simple, cache-friendly collision resolution strategy that works within the fixed-size backing array without dynamic bucket chains.

Verification Method

Test

Uses (parent)

REQ-MAP-001

Code Traces

- `src/juno_map.c:135` (JunoDs_MapSet)

Test Traces

- `tests/test_map.c:365` (test_map_set_single_item)
- `tests/test_map.c:384` (test_map_set_multiple_items)
- `tests/test_map.c:406` (test_map_set_update_existing)
- `tests/test_map.c:430` (test_map_set_with_collision)
- `tests/test_map.c:455` (test_map_set_null_map)
- `tests/test_map.c:663` (test_map_multiple_collisions_linear_probing)
- `tests/test_map.c:714` (test_map_wraparound_probing)
- `tests/test_map.c:741` (test_map_single_element_capacity)
- `tests/test_map.c:776` (test_map_set_get_remove_cycle)

3.8.5. REQ-MAP-005: Map Table Full Rejection

Description

The map shall return JUNO_STATUS_TABLE_FULL_ERROR when no empty slot exists during a Set operation.

Rationale

Explicit rejection on a full table surfaces the capacity limit to the caller rather than silently overwriting existing entries or entering an infinite probe loop.

Verification Method

Test

Uses (parent)

REQ-MAP-004

Code Traces

- `src/juno_map.c:135` (JunoDs_MapSet)

Test Traces

- `tests/test_map.c:465` (test_map_set_table_full)

3.8.6. REQ-MAP-006: Map Get

Description

The map shall retrieve an entry by key using linear probing.

Rationale

Get is the primary read operation; using the same linear probing strategy as Set ensures consistent key lookup behavior.

Verification Method

Test

Uses (parent)

REQ-MAP-001

Code Traces

- `src/juno_map.c:109` (JunoDs_MapGet)

Test Traces

- `tests/test_map.c:490` (test_map_get_existing_item)
- `tests/test_map.c:528` (test_map_get_after_collision)
- `tests/test_map.c:556` (test_map_get_null_map)
- `tests/test_map.c:663` (test_map_multiple_collisions_linear_probing)
- `tests/test_map.c:714` (test_map_wraparound_probing)
- `tests/test_map.c:741` (test_map_single_element_capacity)
- `tests/test_map.c:776` (test_map_set_get_remove_cycle)

3.8.7. REQ-MAP-007: Map Key Not Found

Description

The map shall return JUNO_STATUS_DNE_ERROR when a Get operation does not find the key.

Rationale

Returning a distinct not-found status enables the caller to distinguish between a missing key and a key with a zero/null value.

Verification Method

Test

Uses (parent)

REQ-MAP-006

Code Traces

- `src/juno_map.c:109` (JunoDs_MapGet)

Test Traces

- `tests/test_map.c:515` (test_map_get_nonexistent_item)

3.8.8. REQ-MAP-008: Map Remove

Description

The map shall remove an entry by key using linear probing, resetting the slot.

Rationale

Remove with slot reset ensures that freed slots are returned to the null state expected by the probing algorithm, maintaining map integrity.

Verification Method

Test

Uses (parent)

REQ-MAP-001

Code Traces

- `src/juno_map.c:156` (JunoDs_MapRemove)

Test Traces

- `tests/test_map.c:570` (test_map_remove_existing_item)
- `tests/test_map.c:593` (test_map_remove_nonexistent_item)
- `tests/test_map.c:620` (test_map_remove_after_collision)
- `tests/test_map.c:649` (test_map_remove_null_map)
- `tests/test_map.c:776` (test_map_set_get_remove_cycle)

3.8.9. REQ-MAP-009: Map Vtable Validation

Description

The map module shall verify that its API vtable entries are non-null before each operation.

Rationale

Validates the vtable at runtime to ensure memory safety before dispatching any map operation through function pointers.

Verification Method

Test

Uses (parent)

REQ-SYS-010

Code Traces

- `include/juno/ds/map_api.h:130` (JunoDs_MapVerify)

Test Traces

- `tests/test_map.c:810` (test_map_verify_valid_map)
- `tests/test_map.c:820` (test_map_verify_null_map)

3.8.10. REQ-MAP-010: Map Dependency Validation

Description

The map module shall verify that the backing array, hashable pointer API, and value pointer API dependencies are valid before each operation.

Rationale

Validates all injected dependencies at runtime to catch misconfigured or partially initialized map instances before they produce undefined behavior.

Verification Method

Test

Uses (parent)

REQ-SYS-011

Code Traces

- `include/juno/ds/map_api.h:130` (JunoDs_MapVerify)

Test Traces

- `tests/test_map.c:343` (test_map_init_null_api)
- `tests/test_map.c:351` (test_map_init_zero_capacity)

3.9. Module: MATH

3.9.1. REQ-MATH-001: Math Constants

Description

The math module shall define double-precision constants for π, π/2, π/4, and e.

Rationale

Pre-defined constants avoid repeated magic numbers and ensure consistent precision across all math operations in the library.

Verification Method

Inspection

Uses (parent)

REQ-SYS-001, REQ-SYS-003, REQ-SYS-009

Code Traces

- `include/juno/math/juno_math_constants.h:33` (Pi)

3.9.2. REQ-MATH-002: Vector Types

Description

The math module shall define 2D and 3D vector types in double, float, and int32 precisions.

Rationale

Multiple precision variants allow embedded applications to choose the appropriate trade-off between precision and memory/computation cost for their target platform.

Verification Method

Inspection

Uses (parent)

REQ-SYS-003

Code Traces

- `include/juno/math/juno_vec_types.h:64` (<unknown>)

3.9.3. REQ-MATH-003: Vector Coordinate Access

Description

Each vector type shall support both named Cartesian field access and raw array indexing.

Rationale

Dual access via named fields (x, y, z) and raw array indexing enables both readable application code and loop-based generic algorithms over vector components.

Verification Method

Inspection

Uses (parent)

REQ-MATH-002

Code Traces

- `include/juno/math/juno_vec_types.h:64` (<unknown>)

3.9.4. REQ-MATH-004: Quaternion Type

Description

The math module shall define a real quaternion type with [s, i, j, k] component order in double precision.

Rationale

Quaternions provide singularity-free rotation representation, which is critical for attitude determination and control in embedded navigation and robotics applications.

Verification Method

Inspection

Uses (parent)

REQ-SYS-003

Code Traces

- `include/juno/math/juno_vec_types.h:674` (<unknown>)

3.9.5. REQ-MATH-005: Vector Addition

Description

The math module shall compute component-wise vector addition for each vector type.

Rationale

Vector addition is a fundamental linear algebra operation used in position accumulation, force summation, and signal combination.

Verification Method

Test

Uses (parent)

REQ-MATH-002

Code Traces

- `include/juno/math/juno_vec.h:67` (Juno_Vec2_F64_Add)
- `include/juno/math/juno_vec.h:139` (Juno_Vec2_F32_Add)
- `include/juno/math/juno_vec.h:201` (Juno_Vec2_I32_Add)
- `include/juno/math/juno_vec.h:264` (Juno_Vec3_F64_Add)
- `include/juno/math/juno_vec.h:335` (Juno_Vec3_F32_Add)
- `include/juno/math/juno_vec.h:405` (Juno_Vec3_I32_Add)

Test Traces

- `tests/test_math.c:35` (test_vec2_f64_add)
- `tests/test_math.c:102` (test_vec2_f32_add)
- `tests/test_math.c:169` (test_vec2_i32_add)
- `tests/test_math.c:236` (test_vec3_f64_add)

- `tests/test_math.c:306` (test_vec3_f32_add)
- `tests/test_math.c:376` (test_vec3_i32_add)

3.9.6. REQ-MATH-006: Vector Subtraction

Description

The math module shall compute component-wise vector subtraction for each vector type.

Rationale

Vector subtraction is used for computing displacements, error vectors, and relative quantities between two points or measurements.

Verification Method

Test

Uses (parent)

REQ-MATH-002

Code Traces

- `include/juno/math/juno_vec.h:82` (Juno_Vec2_F64_Sub)
- `include/juno/math/juno_vec.h:151` (Juno_Vec2_F32_Sub)
- `include/juno/math/juno_vec.h:213` (Juno_Vec2_I32_Sub)
- `include/juno/math/juno_vec.h:277` (Juno_Vec3_F64_Sub)
- `include/juno/math/juno_vec.h:348` (Juno_Vec3_F32_Sub)
- `include/juno/math/juno_vec.h:418` (Juno_Vec3_I32_Sub)

Test Traces

- `tests/test_math.c:48` (test_vec2_f64_sub)
- `tests/test_math.c:115` (test_vec2_f32_sub)
- `tests/test_math.c:182` (test_vec2_i32_sub)
- `tests/test_math.c:249` (test_vec3_f64_sub)
- `tests/test_math.c:319` (test_vec3_f32_sub)
- `tests/test_math.c:389` (test_vec3_i32_sub)

3.9.7. REQ-MATH-007: Vector Scalar Multiplication

Description

The math module shall compute component-wise scalar multiplication for each vector type.

Rationale

Scalar multiplication is used for scaling vectors by gain factors, time steps, or unit conversions.

Verification Method

Test

Uses (parent)

REQ-MATH-002

Code Traces

- `include/juno/math/juno_vec.h:96` (Juno_Vec2_F64_Mult)
- `include/juno/math/juno_vec.h:162` (Juno_Vec2_F32_Mult)
- `include/juno/math/juno_vec.h:224` (Juno_Vec2_I32_Mult)
- `include/juno/math/juno_vec.h:289` (Juno_Vec3_F64_Mult)
- `include/juno/math/juno_vec.h:360` (Juno_Vec3_F32_Mult)
- `include/juno/math/juno_vec.h:430` (Juno_Vec3_I32_Mult)

Test Traces

- `tests/test_math.c:61` (test_vec2_f64_mult)
- `tests/test_math.c:128` (test_vec2_f32_mult)
- `tests/test_math.c:195` (test_vec2_i32_mult)
- `tests/test_math.c:262` (test_vec3_f64_mult)
- `tests/test_math.c:332` (test_vec3_f32_mult)
- `tests/test_math.c:402` (test_vec3_i32_mult)

3.9.8. REQ-MATH-008: Vector Dot Product

Description

The math module shall compute the dot product for each vector type.

Rationale

The dot product computes projection and angle-related quantities, which are fundamental to geometry, physics, and signal processing.

Verification Method

Test

Uses (parent)

REQ-MATH-002

Code Traces

- `include/juno/math/juno_vec.h:108` (Juno_Vec2_F64_Dot)
- `include/juno/math/juno_vec.h:173` (Juno_Vec2_F32_Dot)
- `include/juno/math/juno_vec.h:235` (Juno_Vec2_I32_Dot)
- `include/juno/math/juno_vec.h:301` (Juno_Vec3_F64_Dot)
- `include/juno/math/juno_vec.h:372` (Juno_Vec3_F32_Dot)
- `include/juno/math/juno_vec.h:442` (Juno_Vec3_I32_Dot)

Test Traces

- `tests/test_math.c:73` (test_vec2_f64_dot)
- `tests/test_math.c:140` (test_vec2_f32_dot)
- `tests/test_math.c:207` (test_vec2_i32_dot)
- `tests/test_math.c:274` (test_vec3_f64_dot)
- `tests/test_math.c:344` (test_vec3_f32_dot)
- `tests/test_math.c:414` (test_vec3_i32_dot)

3.9.9. REQ-MATH-009: 2D Cross Product

Description

The math module shall compute the pseudoscalar z-component of the 2D cross product.

Rationale

The 2D cross product (pseudoscalar z-component) determines signed area and winding direction, which is useful for 2D geometry and path planning.

Verification Method

Test

Uses (parent)

REQ-MATH-002

Code Traces

- `include/juno/math/juno_vec.h:119` (Juno_Vec2_F64_Cross)
- `include/juno/math/juno_vec.h:182` (Juno_Vec2_F32_Cross)
- `include/juno/math/juno_vec.h:244` (Juno_Vec2_I32_Cross)

Test Traces

- `tests/test_math.c:83` (test_vec2_f64_cross)
- `tests/test_math.c:150` (test_vec2_f32_cross)
- `tests/test_math.c:217` (test_vec2_i32_cross)

3.9.10. REQ-MATH-010: 3D Cross Product

Description

The math module shall compute the 3D cross product returning an orthogonal vector.

Rationale

The 3D cross product computes torque, angular momentum, and surface normals, which are fundamental to 3D kinematics and dynamics.

Verification Method

Test

Uses (parent)

REQ-MATH-002

Code Traces

- `include/juno/math/juno_vec.h:311` (Juno_Vec3_F64_Cross)
- `include/juno/math/juno_vec.h:381` (Juno_Vec3_F32_Cross)
- `include/juno/math/juno_vec.h:451` (Juno_Vec3_I32_Cross)

Test Traces

- `tests/test_math.c:284` (test_vec3_f64_cross)
- `tests/test_math.c:354` (test_vec3_f32_cross)
- `tests/test_math.c:424` (test_vec3_i32_cross)

3.9.11. REQ-MATH-011: Squared L2 Norm

Description

The math module shall compute the squared L2 norm for each vector type.

Rationale

The squared norm avoids a square-root computation, providing an efficient magnitude comparison and normalization building block for resource-constrained targets.

Verification Method

Test

Uses (parent)

REQ-MATH-002

Code Traces

- `include/juno/math/juno_vec.h:129` (Juno_Vec2_F64_L2Norm2)
- `include/juno/math/juno_vec.h:191` (Juno_Vec2_F32_L2Norm2)
- `include/juno/math/juno_vec.h:254` (Juno_Vec2_I32_L2Norm2)
- `include/juno/math/juno_vec.h:325` (Juno_Vec3_F64_L2Norm2)
- `include/juno/math/juno_vec.h:395` (Juno_Vec3_F32_L2Norm2)
- `include/juno/math/juno_vec.h:465` (Juno_Vec3_I32_L2Norm2)

Test Traces

- `tests/test_math.c:93` (test_vec2_f64_L2)
- `tests/test_math.c:160` (test_vec2_f32_L2)
- `tests/test_math.c:227` (test_vec2_i32_L2)
- `tests/test_math.c:297` (test_vec3_f64_L2)
- `tests/test_math.c:367` (test_vec3_f32_L2)
- `tests/test_math.c:437` (test_vec3_i32_L2)

3.9.12. REQ-MATH-012: Quaternion Addition

Description

The math module shall compute component-wise quaternion addition.

Rationale

Quaternion addition is used in interpolation schemes (e.g., SLERP numerator) and filter state updates.

Verification Method

Test

Uses (parent)

REQ-MATH-004

Code Traces

- `include/juno/math/juno_vec.h:476` (Juno_RQuat_F64_Add)

Test Traces

- `tests/test_math.c:446` (test_rquat_f64_add)

3.9.13. REQ-MATH-013: Quaternion Subtraction

Description

The math module shall compute component-wise quaternion subtraction.

Rationale

Quaternion subtraction is used for computing rotation error and difference quaternions in attitude estimation.

Verification Method

Test

Uses (parent)

REQ-MATH-004

Code Traces

- `include/juno/math/juno_vec.h:490` (Juno_RQuat_F64_Sub)

Test Traces

- `tests/test_math.c:458` (test_rquat_f64_sub)

3.9.14. REQ-MATH-014: Quaternion Scalar Multiplication

Description

The math module shall compute component-wise quaternion scalar multiplication.

Rationale

Scalar multiplication is used for scaling quaternion corrections and blending interpolation

weights.

Verification Method

Test

Uses (parent)

REQ-MATH-004

Code Traces

- `include/juno/math/juno_vec.h:503` (Juno_RQuat_F64_Mult)

Test Traces

- `tests/test_math.c:470` (test_rquat_f64_mult_scalar)

3.9.15. REQ-MATH-015: Hamilton Product

Description

The math module shall compute the right-handed Hamilton product of two quaternions.

Rationale

The Hamilton product composes rotations, which is the core operation for attitude propagation and frame transformation in navigation systems.

Verification Method

Test

Uses (parent)

REQ-MATH-004

Code Traces

- `include/juno/math/juno_vec.h:523` (Juno_RQuat_F64_HamProd)

Test Traces

- `tests/test_math.c:482` (test_rquat_f64_hamprod)
- `tests/test_math.c:495` (test_rquat_f64_hamprod_general)

3.9.16. REQ-MATH-016: Quaternion Conjugate

Description

The math module shall compute the conjugate of a quaternion by negating the vector part.

Rationale

The conjugate of a unit quaternion is its inverse rotation, providing an efficient way to compute reverse transformations without division.

Verification Method

Test

Uses (parent)

REQ-MATH-004

Code Traces

- `include/juno/math/juno_vec.h:539` (Juno_RQuat_F64_Conj)

Test Traces

- `tests/test_math.c:509` (test_rquat_f64_conj)

3.9.17. REQ-MATH-017: Quaternion Reciprocal

Description

The math module shall compute the multiplicative inverse of a quaternion as conj(q) divided by the squared norm.

Rationale

The reciprocal generalizes the inverse to non-unit quaternions, which is needed for quaternion division and certain filter update equations.

Verification Method

Test

Uses (parent)

REQ-MATH-016, REQ-MATH-018

Code Traces

- `include/juno/math/juno_vec.h:562` (Juno_RQuat_F64_Recip)

Test Traces

- `tests/test_math.c:538` (test_rquat_f64_recip)

3.9.18. REQ-MATH-018: Quaternion Squared Norm

Description

The math module shall compute the squared L2 norm of a quaternion.

Rationale

The squared norm is the denominator in the reciprocal computation and provides an efficient normalization check without requiring a square root.

Verification Method

Test

Uses (parent)

REQ-MATH-004

Code Traces

- `include/juno/math/juno_vec.h:551` (Juno_RQuat_F64_L2Norm2)

Test Traces

- `tests/test_math.c:520` (test_rquat_f64_l2norm2)
- `tests/test_math.c:529` (test_rquat_f64_l2norm)

3.10. Module: MEMORY

3.10.1. REQ-MEMORY-001: Memory Allocator Interface

Description

The memory module shall define an abstract allocator API with Get, Update, and Put operations.

Rationale

An abstract allocator interface decouples data structures from a specific allocation strategy, enabling the integrator to provide pool-based, slab, or custom allocators suited to their target's memory constraints.

Verification Method

Inspection

Uses (parent)

REQ-SYS-001, REQ-SYS-003, REQ-SYS-004, REQ-SYS-005, REQ-SYS-006, REQ-SYS-009, REQ-SYS-010

Code Traces

- `include/juno/memory/memory_api.h:80` (<unknown>)

3.10.2. REQ-MEMORY-002: Allocator Pointer API Dependency

Description

The memory allocator root shall hold an injected pointer API for internal memory operations.

Rationale

Injecting the pointer API ensures the allocator can perform type-safe copy and reset operations on allocated blocks without depending on a specific pointer implementation.

Verification Method

Inspection

Uses (parent)

REQ-SYS-007, REQ-POINTER-001

Code Traces

- `include/juno/memory/memory_api.h:72` (JUNO_MODULE_ROOT)

3.10.3. REQ-MEMORY-003: Block Allocator Initialization

Description

The block allocator shall initialize from caller-provided backing memory, metadata array, type

size, alignment, and length.

Rationale

Initialization from caller-provided memory ensures the block allocator operates without dynamic allocation and that all backing storage is under integrator control.

Verification Method

Test

Uses (parent)

REQ-SYS-002, REQ-SYS-007, REQ-SYS-008, REQ-SYS-011, REQ-MEMORY-001

Code Traces

- `src/juno_memory_block.c:248` (JunoMemory_BlockInit)

Test Traces

- `tests/test_memory.c:272` (test_invalid_init_parameters)
- `tests/test_memory.c:428` (test_bad_api)

3.10.4. REQ-MEMORY-004: Block Allocator Get

Description

The block allocator shall allocate a block of the configured element size in O(1) time.

Rationale

O(1) allocation time guarantees deterministic performance, which is essential for real-time embedded applications with strict timing budgets.

Verification Method

Test

Uses (parent)

REQ-MEMORY-001

Code Traces

- `src/juno_memory_block.c:65` (Juno_MemoryBlkGet)

Test Traces

- `tests/test_memory.c:83` (test_nominal_single_alloc_and_free)
- `tests/test_memory.c:123` (test_nominal_multiple_alloc_and_free)
- `tests/test_memory.c:185` (test_negative_memory_empty)
- `tests/test_memory.c:234` (test_negative_memory_full)
- `tests/test_memory.c:373` (test_generic_memory_get_put)
- `tests/test_memory.c:406` (test_zero_size_allocation)

3.10.5. REQ-MEMORY-005: Block Allocator Free Block Reuse

Description

The block allocator shall reuse freed blocks via a LIFO free list when available.

Rationale

LIFO free-list reuse maximizes block utilization without fragmentation, keeping the allocator's memory footprint bounded to the initial caller-provided pool.

Verification Method

Test

Uses (parent)

REQ-MEMORY-004

Code Traces

- `src/juno_memory_block.c:65` (Juno_MemoryBlkGet)

Test Traces

- `tests/test_memory.c:123` (test_nominal_multiple_alloc_and_free)

3.10.6. REQ-MEMORY-006: Block Allocator Put

Description

The block allocator shall free a previously allocated block in O(1) time.

Rationale

O(1) deallocation time guarantees deterministic performance and complements the O(1) allocation guarantee.

Verification Method

Test

Uses (parent)

REQ-MEMORY-001

Code Traces

- `src/juno_memory_block.c:158` (Juno_MemoryBlkPut)

Test Traces

- `tests/test_memory.c:83` (test_nominal_single_alloc_and_free)
- `tests/test_memory.c:123` (test_nominal_multiple_alloc_and_free)
- `tests/test_memory.c:373` (test_generic_memory_get_put)

3.10.7. REQ-MEMORY-007: Block Allocator Double Free Rejection

Description

The block allocator shall reject double frees with an error status.

Rationale

Detecting double frees at runtime prevents free-list corruption that would otherwise cause silent data corruption or unbounded allocation behavior.

Verification Method

Test

Uses (parent)

REQ-MEMORY-006

Code Traces

- `src/juno_memory_block.c:158` (Juno_MemoryBlkPut)

Test Traces

- `tests/test_memory.c:290` (test_double_free)

3.10.8. REQ-MEMORY-008: Block Allocator Invalid Address Rejection

Description

The block allocator shall reject frees of invalid or misaligned addresses with an error status.

Rationale

Rejecting invalid or misaligned addresses prevents corruption of the free list and ensures that only addresses within the original backing memory are returned to the pool.

Verification Method

Test

Uses (parent)

REQ-MEMORY-006

Code Traces

- `src/juno_memory_block.c:158` (Juno_MemoryBlkPut)

Test Traces

- `tests/test_memory.c:319` (test_free_unallocated)
- `tests/test_memory.c:464` (test_invalid_size_and_addr)

3.10.9. REQ-MEMORY-009: Block Allocator Update

Description

The block allocator shall adjust a descriptor's size in-place up to the configured element size.

Rationale

In-place size adjustment allows a descriptor to shrink or grow within its fixed block without triggering a new allocation or data copy, which is useful for variable-length payloads within fixed-size slots.

Verification Method

Test

Uses (parent)

REQ-MEMORY-001

Code Traces

- `src/juno_memory_block.c:139` (Juno_MemoryBlkUpdate)

Test Traces

- `tests/test_memory.c:341` (test_update_memory)

3.10.10. REQ-MEMORY-010: Block Allocator No Memory Move

Description

The block allocator Update shall not move memory when adjusting descriptor size.

Rationale

Guaranteeing no memory movement during Update ensures that existing pointers to the block remain valid, which is critical for data structures that hold references to allocated blocks.

Verification Method

Test

Uses (parent)

REQ-MEMORY-009

Code Traces

- `src/juno_memory_block.c:139` (Juno_MemoryBlkUpdate)

Test Traces

- `tests/test_memory.c:341` (test_update_memory)

3.11. Module: MODULE

3.11.1. REQ-MODULE-001: Module Root Structure

Description

The module system shall define a module root struct containing an API vtable pointer.

Rationale

The module root struct is the foundation of the vtable-based dependency injection system, enabling polymorphic dispatch across all LibJuno modules.

Verification Method

Inspection

Uses (parent)

REQ-SYS-001, REQ-SYS-003, REQ-SYS-006, REQ-SYS-009

Code Traces

- `include/juno/module.h:128` (<unknown>)

3.11.2. REQ-MODULE-002: Module Root Failure Handler

Description

The module root shall contain a failure handler pointer and user data pointer.

Rationale

Provides a diagnostic callback mechanism so integrators can observe failures without altering control flow, which is essential for debugging on targets with limited instrumentation.

Verification Method

Inspection

Uses (parent)

REQ-STATUS-003

Code Traces

- `include/juno/module.h:128` (<unknown>)

3.11.3. REQ-MODULE-003: Module Derivation

Description

Derived module structs shall embed the root as their first member, enabling safe upcasting to the root type.

Rationale

Struct embedding with the root as the first member enables safe upcasting in C, providing polymorphism without dynamic allocation or language extensions.

Verification Method

Inspection

Uses (parent)

REQ-SYS-006

Code Traces

- `include/juno/module.h:160` (<unknown>)

3.11.4. REQ-MODULE-004: Module Union

Description

A module union shall contain the root and all derivation types for polymorphic storage.

Rationale

The union enables caller-allocated polymorphic storage that is correctly sized for any derivation, without requiring the caller to know which concrete type will be used at compile time.

Verification Method

Inspection

Uses (parent)

REQ-SYS-006

Code Traces

- `include/juno/module.h:111` (<unknown>)

3.11.5. REQ-MODULE-005: Result Type

Description

The module system shall define a result type combining a status code with a typed success payload.

Rationale

Combines a status code with a typed payload in a single return value, enabling functions to communicate both success data and error conditions without out-parameters or global state.

Verification Method

Inspection

Uses (parent)

REQ-SYS-004

Code Traces

- `include/juno/module.h:197` (<unknown>)

3.11.6. REQ-MODULE-006: Option Type

Description

The module system shall define an option type combining a presence flag with a typed payload.

Rationale

Provides explicit representation of optional values, eliminating sentinel-value conventions and making the presence or absence of a value type-safe and self-documenting.

Verification Method

Inspection

Uses (parent)

REQ-SYS-004

Code Traces

- `include/juno/module.h:241` (<unknown>)

3.12. Module: MTX

3.12.1. REQ-MTX-001: Mutex Root Structure

Description

The mutex root shall embed a module root with an API vtable pointer.

Rationale

An abstract mutex interface decouples synchronization logic from platform-specific locking primitives (RTOS mutexes, spinlocks, etc.), enabling portability and testability.

Verification Method

Inspection

Uses (parent)

REQ-SYS-001, REQ-SYS-002, REQ-SYS-003, REQ-SYS-006, REQ-SYS-007, REQ-SYS-009

Code Traces

- `include/juno/mp/mtx_api.h:44` (JUNO_MODULE_ROOT)

3.12.2. REQ-MTX-002: TryLock Operation

Description

The mutex API vtable shall define a TryLock operation that attempts to acquire the lock and returns immediately with a boolean result.

Rationale

TryLock enables non-blocking lock acquisition, which is essential for real-time tasks that cannot afford to block indefinitely on a contended resource.

Verification Method

Inspection

Uses (parent)

REQ-MTX-001

Code Traces

- `include/juno/mp/mtx_api.h:50` (JUNO_RESULT_BOOL_T)

3.12.3. REQ-MTX-003: Lock Operation

Description

The mutex API vtable shall define a Lock operation that blocks until the lock is acquired.

Rationale

Blocking Lock provides the standard synchronization primitive for tasks that can afford to wait, complementing the non-blocking TryLock.

Verification Method

Inspection

Uses (parent)

REQ-MTX-001

Code Traces

- `include/juno/mp/mtx_api.h:53` (JUNO_STATUS_T)

3.12.4. REQ-MTX-004: Free Operation

Description

The mutex API vtable shall define a Free operation that releases the lock.

Rationale

Free releases the lock, enabling other tasks to acquire it; without Free the mutex would be permanently held.

Verification Method

Inspection

Uses (parent)

REQ-MTX-001

Code Traces

- `include/juno/mp/mtx_api.h:56` (JUNO_STATUS_T)

3.12.5. REQ-MTX-005: Mutex Vtable Validation

Description

The mutex module shall verify that its API vtable and all function pointers within it are non-NULL.

Rationale

Validates the vtable at runtime to ensure memory safety before dispatching any mutex operation through function pointers.

Verification Method

Test

Uses (parent)

REQ-SYS-010

Code Traces

- `include/juno/mp/mtx_api.h:61` (JunoMp_MtxVerify)

Test Traces

- `tests/test_mtx.c:77` (test_mtx_verify_valid)
- `tests/test_mtx.c:84` (test_mtx_verify_null_root)
- `tests/test_mtx.c:91` (test_mtx_verify_null_api)
- `tests/test_mtx.c:99` (test_mtx_verify_null_trylock)

3.13. Module: POINTER

3.13.1. REQ-POINTER-001: Pointer Descriptor Structure

Description

The pointer module shall define a pointer trait (JUNO_POINTER_T) containing an API vtable pointer, memory address, size, and alignment.

Rationale

A fat pointer descriptor enables type-safe, size-aware memory access so that data structures can operate on arbitrary element types without unsafe void-pointer arithmetic.

Verification Method

Inspection

Uses (parent)

REQ-SYS-001, REQ-SYS-003, REQ-SYS-004, REQ-SYS-005, REQ-SYS-006, REQ-SYS-009

Code Traces

- `include/juno/memory/pointer_api.h:55` (JUNO_TRAIT_ROOT)

3.13.2. REQ-POINTER-002: Pointer Copy

Description

The pointer API shall copy the value referenced by a source descriptor into a destination descriptor's storage location.

Rationale

Copy is the fundamental value-transfer operation for data structures, enabling element insertion and extraction without exposing raw memory addresses to callers.

Verification Method

Test

Uses (parent)

REQ-POINTER-001

Code Traces

- `include/juno/memory/pointer_api.h:156` (JunoMemory_PointerApiVerify)

Test Traces

- `tests/test_map.c:833` (test_map_entry_copy_nominal)
- `tests/test_map.c:849` (test_map_entry_copy_null_dest)
- `tests/test_pointer.c:89` (test_pointer_copy_success)
- `tests/test_pointer.c:100` (test_pointer_copy_null_src_addr)
- `tests/test_stack.c:1032` (test_stack_pointer_copy_invalid_dest)
- `tests/test_stack.c:1043` (test_stack_pointer_copy_invalid_src)

3.13.3. REQ-POINTER-003: Pointer Reset

Description

The pointer API shall reset the memory at a pointer descriptor to a defined empty state.

Rationale

Reset provides a deterministic way to clear memory slots, ensuring that data structures can return elements to a known-empty state without leaking stale data.

Verification Method

Test

Uses (parent)

REQ-POINTER-001

Code Traces

- `include/juno/memory/pointer_api.h:156` (JunoMemory_PointerApiVerify)

Test Traces

- `tests/test_map.c:860` (test_map_entry_reset_nominal)
- `tests/test_pointer.c:113` (test_pointer_reset_success)
- `tests/test_pointer.c:123` (test_pointer_reset_null_addr)
- `tests/test_stack.c:1054` (test_stack_pointer_reset_invalid)
- `tests/test_stack.c:1062` (test_stack_pointer_reset_valid)

3.13.4. REQ-POINTER-004: Pointer Descriptor Validation

Description

The pointer module shall validate that a pointer descriptor's API, address, and size are non-null/non-zero.

Rationale

Validates descriptor integrity at runtime to ensure memory safety before any copy or reset operation dereferences the address.

Verification Method

Test

Uses (parent)

REQ-SYS-008, REQ-SYS-010, REQ-POINTER-001

Code Traces

- `include/juno/memory/pointer_api.h:188` (JunoMemory_PointerVerify)

Test Traces

- `tests/test_pointer.c:135` (test_pointer_verify_valid)
- `tests/test_pointer.c:143` (test_pointer_verify_null_api)
- `tests/test_pointer.c:151` (test_pointer_verify_null_addr)
- `tests/test_pointer.c:159` (test_pointer_verify_zero_size)

3.13.5. REQ-POINTER-005: Pointer Type Verification

Description

The pointer module shall verify that a pointer descriptor's size and alignment match an expected type.

Rationale

Verifying size and alignment against an expected type catches type mismatches at runtime, preventing silent memory corruption when descriptors are used with the wrong element type.

Verification Method

Test

Uses (parent)

REQ-SYS-008, REQ-POINTER-004

Code Traces

- `include/juno/memory/pointer_api.h:129` (JunoMemory_PointerVerify)

Test Traces

- `tests/test_pointer.c:172` (test_pointer_verify_type_match)
- `tests/test_pointer.c:180` (test_pointer_verify_type_wrong_size)
- `tests/test_pointer.c:189` (test_pointer_verify_type_wrong_alignment)

3.14. Module: QUEUE

3.14.1. REQ-QUEUE-001: Queue Initialization

Description

The queue module shall initialize from a caller-provided backing array.

Rationale

FIFO queues are essential for message passing, event buffering, and inter-module communication in embedded systems; initializing from a caller-provided array satisfies the zero-allocation constraint.

Verification Method

Test

Uses (parent)

REQ-SYS-001, REQ-SYS-002, REQ-SYS-003, REQ-SYS-004, REQ-SYS-005, REQ-SYS-006, REQ-SYS-009, REQ-SYS-010, REQ-ARRAY-001

Code Traces

- `src/juno_buff_queue.c:106` (JunoDs_QueueInit)
- `include/juno/ds/queue_api.h:116` (JunoDs_QueueVerify)

Test Traces

- `tests/test_queue.c:209` (test_queue_init_nominal)
- `tests/test_queue.c:220` (test_queue_init_null_queue)
- `tests/test_queue.c:230` (test_queue_init_null_array)
- `tests/test_queue.c:237` (test_queue_init_zero_capacity)
- `tests/test_queue.c:729` (test_queue_verify_valid_queue)
- `tests/test_queue.c:739` (test_queue_verify_null_queue)
- `tests/test_queue.c:746` (test_queue_verify_invalid_api)
- `tests/test_queue.c:759` (test_queue_api_verify_valid)
- `tests/test_queue.c:769` (test_queue_api_verify_null)
- `tests/test_queue.c:776` (test_queue_api_verify_missing_functions)

3.14.2. REQ-QUEUE-002: Queue Initial State

Description

The queue shall set start index and length to zero on initialization.

Rationale

Zeroing the start index and length on initialization establishes a known-empty state, ensuring the queue is immediately usable without undefined behavior from uninitialized fields.

Verification Method

Test

Uses (parent)

REQ-SYS-002, REQ-SYS-007, REQ-SYS-008, REQ-SYS-011, REQ-QUEUE-001

Code Traces

- `src/juno_buff_queue.c:106` (JunoDs_QueueInit)

Test Traces

- `tests/test_queue.c:209` (test_queue_init_nominal)

3.14.3. REQ-QUEUE-003: Queue Enqueue

Description

The queue shall copy an element to the back of the circular buffer in O(1) time.

Rationale

O(1) enqueue via a circular buffer provides deterministic timing for real-time message producers.

Verification Method

Test

Uses (parent)

REQ-QUEUE-001

Code Traces

- `src/juno_buff_queue.c:24` (JunoDs_QueuePush)

Test Traces

- `tests/test_queue.c:249` (test_queue_push_single_item)
- `tests/test_queue.c:266` (test_queue_push_multiple_items)
- `tests/test_queue.c:290` (test_queue_push_to_full_capacity)
- `tests/test_queue.c:331` (test_queue_push_null_queue)
- `tests/test_queue.c:341` (test_queue_push_invalid_pointer)
- `tests/test_queue.c:554` (test_queue_circular_wraparound)
- `tests/test_queue.c:605` (test_queue_wraparound_at_boundary)
- `tests/test_queue.c:642` (test_queue_stress_push_pop_cycles)
- `tests/test_queue.c:673` (test_queue_interleaved_operations)
- `tests/test_queue.c:803` (test_queue_single_element_capacity)
- `tests/test_queue.c:831` (test_queue_boundary_index_calculation)
- `tests/test_queue.c:860` (test_queue_data_integrity_after_operations)

3.14.4. REQ-QUEUE-004: Queue Full Rejection

Description

The queue shall return JUNO_STATUS_INVALID_SIZE_ERROR when enqueue is attempted on a full queue.

Rationale

Explicit rejection on a full queue surfaces the overflow condition to the caller rather than silently dropping or overwriting data.

Verification Method

Test

Uses (parent)

REQ-QUEUE-003

Code Traces

- `src/juno_buff_queue.c:24` (JunoDs_QueuePush)

Test Traces

- `tests/test_queue.c:308` (test_queue_push_overflow)

3.14.5. REQ-QUEUE-005: Queue Dequeue

Description

The queue shall remove and copy out the front element in O(1) time.

Rationale

O(1) dequeue via a circular buffer provides deterministic timing for real-time message consumers.

Verification Method

Test

Uses (parent)

REQ-QUEUE-001

Code Traces

- `src/juno_buff_queue.c:50` (JunoDs_QueuePop)

Test Traces

- `tests/test_queue.c:356` (test_queue_pop_single_item)
- `tests/test_queue.c:379` (test_queue_pop_multiple_items_fifo_order)
- `tests/test_queue.c:421` (test_queue_pop_null_queue)
- `tests/test_queue.c:430` (test_queue_pop_invalid_pointer)
- `tests/test_queue.c:554` (test_queue_circular_wraparound)
- `tests/test_queue.c:605` (test_queue_wraparound_at_boundary)
- `tests/test_queue.c:642` (test_queue_stress_push_pop_cycles)
- `tests/test_queue.c:673` (test_queue_interleaved_operations)
- `tests/test_queue.c:803` (test_queue_single_element_capacity)
- `tests/test_queue.c:860` (test_queue_data_integrity_after_operations)

3.14.6. REQ-QUEUE-006: Queue Empty Dequeue Rejection

Description

The queue shall return JUNO_STATUS_ERR when dequeue is attempted on an empty queue.

Rationale

Explicit rejection on an empty queue surfaces the underflow condition rather than returning stale or uninitialized data.

Verification Method

Test

Uses (parent)

REQ-QUEUE-005

Code Traces

- `src/juno_buff_queue.c:50` (JunoDs_QueuePop)

Test Traces

- `tests/test_queue.c:409` (test_queue_pop_empty_queue)

3.14.7. REQ-QUEUE-007: Queue Peek

Description

The queue shall return a pointer descriptor to the front element without removing it in O(1) time.

Rationale

Peek allows consumers to inspect the next element without consuming it, which is useful for conditional dequeue logic and priority-based processing.

Verification Method

Test

Uses (parent)

REQ-QUEUE-001

Code Traces

- `src/juno_buff_queue.c:78` (JunoDs_QueuePeek)

Test Traces

- `tests/test_queue.c:452` (test_queue_peek_single_item)
- `tests/test_queue.c:478` (test_queue_peek_multiple_times)
- `tests/test_queue.c:515` (test_queue_peek_null_queue)
- `tests/test_queue.c:522` (test_queue_peek_after_pop)

3.14.8. REQ-QUEUE-008: Queue Empty Peek Rejection

Description

The queue shall return JUNO_STATUS_INVALID_SIZE_ERROR when peek is attempted on an empty queue.

Rationale

Explicit rejection on empty peek prevents the caller from receiving a descriptor to uninitialized or stale memory.

Verification Method

Test

Uses (parent)

REQ-QUEUE-007

Code Traces

- `src/juno_buff_queue.c:78` (JunoDs_QueuePeek)

Test Traces

- `tests/test_queue.c:505` (test_queue_peek_empty_queue)

3.15. Module: SB

3.15.1. REQ-SB-001: Broker Root Structure

Description

The broker root shall hold a pipe registry pointer, registry length, and registry capacity.

Rationale

The broker root holds the pipe registry that enables publish/subscribe message routing, providing the core data needed to fan out messages to subscribers.

Verification Method

Inspection

Uses (parent)

REQ-SYS-001, REQ-SYS-003, REQ-SYS-004, REQ-SYS-005, REQ-SYS-006, REQ-SYS-009

Code Traces

- `include/juno/sb/broker_api.h:79` (JUNO_MODULE_ROOT)

3.15.2. REQ-SB-002: Broker API Interface

Description

The broker API vtable shall define Publish and RegisterSubscriber operations.

Rationale

Publish and RegisterSubscriber are the minimal operations needed for a publish/subscribe software bus: producers publish messages and consumers register pipes to receive them.

Verification Method

Inspection

Uses (parent)

REQ-SB-001

Code Traces

- `include/juno/sb/broker_api.h:89` (<unknown>)

3.15.3. REQ-SB-003: Broker Initialization

Description

The broker shall initialize from a caller-provided pipe registry array and capacity.

Rationale

Initialization from a caller-provided registry array ensures the broker operates without dynamic allocation and that the registry capacity is under integrator control.

Verification Method

Test

Uses (parent)

REQ-SYS-002, REQ-SYS-007, REQ-SYS-008, REQ-SB-001

Code Traces

- `src/juno_broker.c:35` (JunoSb_BrokerInit)

Test Traces

- `tests/test_sb.c:190` (test_sb_broker_init_success)
- `tests/test_sb.c:203` (test_sb_broker_init_null_broker)
- `tests/test_sb.c:212` (test_sb_broker_init_null_registry)

3.15.4. REQ-SB-004: Pipe Derivation

Description

A pipe shall derive from the queue module root with an additional message ID field.

Rationale

Deriving a pipe from the queue root reuses the existing FIFO implementation and adds only the message ID field needed for topic-based routing, minimizing code duplication.

Verification Method

Inspection

Uses (parent)

REQ-MODULE-003, REQ-QUEUE-001

Code Traces

- `include/juno/sb/broker_api.h:72` (JUNO_MODULE_DERIVE)

3.15.5. REQ-SB-005: Pipe Initialization

Description

A pipe shall initialize with a message ID and a backing queue array.

Rationale

Initializing a pipe with a message ID and backing queue establishes the subscription topic and buffer in a single step, with all memory caller-owned.

Verification Method

Test

Uses (parent)

REQ-SB-004, REQ-SYS-002

Code Traces

- `include/juno/sb/broker_api.h:138` (JunoSb_PipeInit)

Test Traces

- `tests/test_sb.c:225` (test_sb_pipe_init_success)
- `tests/test_sb.c:234` (test_sb_pipe_init_null_pipe)

3.15.6. REQ-SB-006: Publish Fan-Out

Description

The broker shall enqueue a message into every registered pipe whose message ID matches the published message ID.

Rationale

Fan-out delivery to all matching pipes is the defining behavior of a publish/subscribe bus, decoupling producers from consumers by message topic.

Verification Method

Test

Uses (parent)

REQ-SB-002

Code Traces

- `src/juno_broker.c:50` (Publish)

Test Traces

- `tests/test_sb.c:280` (test_sb_publish_fan_out)

3.15.7. REQ-SB-007: Publish Non-Matching Skip

Description

The broker shall skip pipes whose message ID does not match the published message ID.

Rationale

Skipping non-matching pipes ensures that subscribers only receive messages for their registered topic, preventing cross-contamination between message channels.

Verification Method

Test

Uses (parent)

REQ-SB-006

Code Traces

- `src/juno_broker.c:50` (Publish)

Test Traces

- `tests/test_sb.c:309` (test_sb_publish_skip_non_matching)

3.15.8. REQ-SB-008: Register Subscriber

Description

The broker shall append a pipe pointer to the registry on registration.

Rationale

Appending a pipe to the registry is the mechanism by which consumers subscribe to the software bus, establishing the routing table for future publish operations.

Verification Method

Test

Uses (parent)

REQ-SB-002

Code Traces

- `src/juno_broker.c:75` (RegisterSubscriber)

Test Traces

- `tests/test_sb.c:246` (test_sb_register_subscriber)

3.15.9. REQ-SB-009: Registry Full Rejection

Description

The broker shall return an error status when registration is attempted on a full registry.

Rationale

Explicit rejection on a full registry surfaces the capacity limit to the caller rather than silently failing to register a subscriber.

Verification Method

Test

Uses (parent)

REQ-SB-008

Code Traces

- `src/juno_broker.c:75` (RegisterSubscriber)

Test Traces

- `tests/test_sb.c:259` (test_sb_register_full_rejection)

3.15.10. REQ-SB-010: Broker Vtable Validation

Description

The broker shall verify that its API vtable entries are non-null before each operation.

Rationale

Validates the vtable and registry dependency at runtime to ensure memory safety before dispatching any broker operation.

Verification Method

Test

Uses (parent)

REQ-SYS-010, REQ-SYS-011

Code Traces

- `include/juno/sb/broker_api.h:113` (JunoSb_BrokerVerify)

Test Traces

- `tests/test_sb.c:336` (test_sb_broker_verify_null_api)

3.15.11. REQ-SB-011: Broker Registry Validation

Description

The broker shall verify that its pipe registry pointer and capacity are valid before each operation.

Rationale

Validates the registry pointer and capacity at runtime to catch uninitialized or misconfigured broker instances before they produce undefined behavior.

Verification Method

Test

Uses (parent)

REQ-SYS-011

Code Traces

- `include/juno/sb/broker_api.h:113` (JunoSb_BrokerVerify)

Test Traces

- `tests/test_sb.c:346` (test_sb_broker_verify_null_registry)
- `tests/test_sb.c:356` (test_sb_broker_verify_zero_capacity)

3.15.12. REQ-SB-012: Pipe Validation

Description

The broker shall verify each pipe via the queue verification before publishing to it.

Rationale

Verifying each pipe before publishing to it prevents the broker from enqueuing messages into uninitialized or corrupted queue instances.

Verification Method

Test

Uses (parent)

REQ-SB-006, REQ-SYS-008

Code Traces

- `include/juno/sb/broker_api.h:125` (JunoSb_PipeVerify)

Test Traces

- `tests/test_sb.c:366` (test_sb_pipe_verify_valid)
- `tests/test_sb.c:374` (test_sb_pipe_verify_null)

3.15.13. REQ-SB-013: Single Thread Scope

Description

The broker shall operate within a single thread without thread-safety mechanisms.

Rationale

Single-thread scope avoids the complexity and overhead of synchronization primitives, which aligns with the deterministic, freestanding design philosophy; thread safety can be layered externally via the mutex module if needed.

Verification Method

Inspection

Code Traces

- `include/juno/sb/broker_api.h:50` (<unknown>)

3.16. Module: SCH

3.16.1. REQ-SCH-001: Scheduler Root Structure

Description

The scheduler root shall hold a time source pointer, a flattened schedule table, apps-per-minor-frame count, minor-frame count, and minor-frame period.

Rationale

The scheduler root holds all configuration needed to execute a time-triggered schedule: the time source for pacing, and the table layout defining which applications run in which minor frames.

Verification Method

Inspection

Uses (parent)

REQ-SYS-001, REQ-SYS-002, REQ-SYS-003, REQ-SYS-006, REQ-SYS-007, REQ-SYS-009, REQ-SYS-010

Code Traces

- `include/juno/sch/juno_sch_api.h:61` (JUNO_MODULE_ROOT)

3.16.2. REQ-SCH-002: Scheduler API Interface

Description

The scheduler API vtable shall define Execute, GetMinorFramePeriod, and GetMajorFramePeriod operations.

Rationale

Execute, GetMinorFramePeriod, and GetMajorFramePeriod are the minimal API surface needed to run and query a time-triggered schedule.

Verification Method

Inspection

Uses (parent)

REQ-SCH-001

Code Traces

- `include/juno/sch/juno_sch_api.h:70` (<unknown>)

3.16.3. REQ-SCH-003: Schedule Table Layout

Description

The schedule table shall be a flattened 2D array of application pointers with dimensions minor-

frame count by apps-per-minor-frame.

Rationale

A flattened 2D array avoids pointer-to-pointer indirection and dynamic allocation, keeping the schedule table in a single contiguous caller-provided buffer.

Verification Method

Inspection

Uses (parent)

REQ-SCH-001, REQ-SYS-002

Code Traces

- `include/juno/sch/juno_sch_api.h:56` (<unknown>)

3.16.4. REQ-SCH-004: Execute Major Frame

Description

The scheduler Execute operation shall invoke each application's OnProcess hook across all minor frames in one major frame.

Rationale

Executing all minor frames in sequence completes one major frame, which is the standard time-triggered scheduling pattern used in flight software and real-time control systems.

Verification Method

Test

Uses (parent)

REQ-SCH-002, REQ-APP-003

Code Traces

- `include/juno/sch/juno_sch_api.h:74` (JUNO_STATUS_T)

Test Traces

- `tests/test_sch.c:153` (test_sch_execute_invokes_all_apps)
- `tests/test_sch.c:162` (test_sch_execute_skips_null_slot)

3.16.5. REQ-SCH-005: Minor Frame Period Query

Description

The scheduler shall return the configured minor frame period as a timestamp.

Rationale

Exposing the minor frame period allows applications and diagnostics to query the scheduler's timing granularity without accessing internal fields.

Verification Method

Test

Uses (parent)

REQ-SCH-002

Code Traces

- `include/juno/sch/juno_sch_api.h:77` (JUNO_TIMESTAMP_RESULT_T)

Test Traces

- `tests/test_sch.c:177` (test_sch_get_minor_frame_period)

3.16.6. REQ-SCH-006: Major Frame Period Computation

Description

The scheduler shall compute the major frame period as the product of the minor frame period and the minor-frame count.

Rationale

The major frame period is a derived quantity that callers need for cycle-level timing and telemetry; computing it from the minor frame period and count ensures consistency.

Verification Method

Test

Uses (parent)

REQ-SCH-005

Code Traces

- `include/juno/sch/juno_sch_api.h:80` (JUNO_TIMESTAMP_RESULT_T)

Test Traces

- `tests/test_sch.c:190` (test_sch_get_major_frame_period)

3.16.7. REQ-SCH-007: Time Source Dependency

Description

The scheduler shall use an injected time module for timing operations.

Rationale

Injecting the time source decouples the scheduler from any specific platform clock, enabling both real hardware timing and deterministic test clocks.

Verification Method

Inspection

Uses (parent)

REQ-SYS-007, REQ-TIME-002

Code Traces

- `include/juno/sch/juno_sch_api.h:61` (JUNO_MODULE_ROOT)

3.17. Module: SM

3.17.1. REQ-SM-001: SM Root Structure

Description

The state machine root shall hold a pointer to the current state.

Rationale

The state machine root holds the current state pointer, which is the minimal state needed to drive a state machine without dynamic allocation or global state.

Verification Method

Inspection

Uses (parent)

REQ-SYS-001, REQ-SYS-003, REQ-SYS-004, REQ-SYS-005, REQ-SYS-006, REQ-SYS-009, REQ-SYS-010

Code Traces

- `include/juno/sm/sm_api.h:76` (JUNO_MODULE_ROOT)

3.17.2. REQ-SM-002: State Root Structure

Description

Each state shall embed a module root with an API vtable, a pointer to the owning SM, and a next-state option.

Rationale

Embedding a module root in each state enables polymorphic dispatch of state actions through the vtable, allowing different state implementations to share the same interface.

Verification Method

Inspection

Uses (parent)

REQ-SYS-006, REQ-MODULE-003

Code Traces

- `include/juno/sm/sm_api.h:59` (JUNO_MODULE_ROOT)

3.17.3. REQ-SM-003: State API Interface

Description

Each state API vtable shall define StateAction, ShouldExit, and ResetState operations.

Rationale

StateAction, ShouldExit, and ResetState define the minimal lifecycle hooks needed to execute, terminate, and reinitialize a state, keeping the interface simple and composable.

Verification Method

Inspection

Uses (parent)

REQ-SM-002

Code Traces

- `include/juno/sm/sm_api.h:65` (JUNO_STATUS_T)

3.17.4. REQ-SM-004: SM Initialization

Description

The state machine shall initialize with a start state and failure handler.

Rationale

Initializing with a start state and failure handler establishes the SM in a valid, runnable configuration with diagnostic support from the first invocation.

Verification Method

Test

Uses (parent)

REQ-SYS-002, REQ-SYS-007, REQ-SM-001

Code Traces

- `include/juno/sm/sm_api.h:126` (JunoSm_Init)

Test Traces

- `tests/test_sm.c:91` (test_sm_init_success)
- `tests/test_sm.c:103` (test_sm_init_null_root)
- `tests/test_sm.c:110` (test_sm_init_null_start_state)

3.17.5. REQ-SM-005: State Initialization

Description

A state shall initialize with a link to its owning SM, an API vtable, and an optional next state.

Rationale

Linking a state to its owning SM enables the state to trigger transitions and query SM-level context without global references.

Verification Method

Test

Uses (parent)

REQ-SM-002, REQ-SYS-007

Code Traces

- `include/juno/sm/sm_api.h:104` (JunoSm_StateInit)

Test Traces

- `tests/test_sm.c:121` (test_sm_state_init_with_next)
- `tests/test_sm.c:136` (test_sm_state_init_without_next)
- `tests/test_sm.c:148` (test_sm_state_init_null_state)
- `tests/test_sm.c:156` (test_sm_state_init_null_api)

3.17.6. REQ-SM-006: Get Current State

Description

The state machine shall return the current state pointer.

Rationale

Exposing the current state pointer allows external logic to inspect SM status, which is useful for diagnostics, telemetry, and conditional behavior.

Verification Method

Test

Uses (parent)

REQ-SM-001

Code Traces

- `include/juno/sm/sm_api.h:139` (JunoSm_GetCurrentState)

Test Traces

- `tests/test_sm.c:168` (test_sm_get_current_state)
- `tests/test_sm.c:179` (test_sm_get_current_state_null_root)

3.17.7. REQ-SM-007: State Transition

Description

The state machine shall transition to the next state when the current state's next-state option is present.

Rationale

Option-driven transitions decouple transition logic from the SM engine; a state signals a transition by setting the next-state option, keeping the transition protocol explicit and deterministic.

Verification Method

Test

Uses (parent)

REQ-SM-001

Code Traces

- `include/juno/sm/sm_api.h:151` (JunoSm_TransitionState)

Test Traces

- `tests/test_sm.c:190` (test_sm_transition_to_next)
- `tests/test_sm.c:220` (test_sm_transition_null_root)

3.17.8. REQ-SM-008: No Transition When None

Description

The state machine shall remain in the current state when the next-state option is absent.

Rationale

Remaining in the current state when no transition is requested is the safe default, preventing unintended state changes due to uninitialized option fields.

Verification Method

Test

Uses (parent)

REQ-SM-007

Code Traces

- `include/juno/sm/sm_api.h:151` (JunoSm_TransitionState)

Test Traces

- `tests/test_sm.c:207` (test_sm_transition_stays_when_none)

3.17.9. REQ-SM-009: SM Verification

Description

The state machine shall verify that its root and current state pointer are non-NULL.

Rationale

Verifying root and state pointer validity at runtime ensures memory safety before the SM engine dereferences any internal pointers.

Verification Method

Test

Uses (parent)

REQ-SYS-008

Code Traces

- `include/juno/sm/sm_api.h:83` (JunoSm_Verify)

Test Traces

- `tests/test_sm.c:231` (test_sm_verify_valid)
- `tests/test_sm.c:241` (test_sm_verify_null_root)
- `tests/test_sm.c:248` (test_sm_verify_null_current_state)

3.17.10. REQ-SM-010: State Verification

Description

Each state shall verify that its API vtable and owning SM pointer are non-NULL.

Rationale

Validating each state's vtable and owning-SM pointer at runtime catches misconfigured states before they produce undefined behavior through invalid dispatch or null dereference.

Verification Method

Test

Uses (parent)

REQ-SYS-010, REQ-SYS-011

Code Traces

- `include/juno/sm/sm_api.h:91` (JunoSm_StateVerify)

Test Traces

- `tests/test_sm.c:260` (test_sm_state_verify_valid)
- `tests/test_sm.c:270` (test_sm_state_verify_null_state)
- `tests/test_sm.c:277` (test_sm_state_verify_null_api)
- `tests/test_sm.c:286` (test_sm_state_verify_null_sm)

3.17.11. REQ-SM-011: Next State Pointer Verification

Description

Each state shall verify that the next-state pointer is non-NULL when the next-state option is marked present.

Rationale

A present option with a NULL pointer would cause a null dereference during transition; validating this precondition prevents that undefined behavior.

Verification Method

Test

Uses (parent)

REQ-SYS-008

Code Traces

- `include/juno/sm/sm_api.h:91` (JunoSm_StateVerify)

Test Traces

- `tests/test_sm.c:295` (test_sm_state_verify_next_null_when_some)
- `tests/test_sm.c:306` (test_sm_state_verify_next_valid_when_some)

3.18. Module: STACK

3.18.1. REQ-STACK-001: Stack Initialization

Description

The stack module shall initialize from a caller-provided backing array.

Rationale

LIFO stacks are essential for undo buffers, expression evaluation, and depth-first traversal in embedded systems; initializing from a caller-provided array satisfies the zero-allocation constraint.

Verification Method

Test

Uses (parent)

REQ-SYS-001, REQ-SYS-002, REQ-SYS-003, REQ-SYS-004, REQ-SYS-005, REQ-SYS-006, REQ-SYS-009, REQ-SYS-010, REQ-ARRAY-001

Code Traces

- `src/juno_buff_stack.c:101` (JunoDs_StackInit)

Test Traces

- `tests/test_stack.c:211` (test_stack_init_nominal)
- `tests/test_stack.c:221` (test_stack_init_null_stack)
- `tests/test_stack.c:228` (test_stack_init_null_api)
- `tests/test_stack.c:236` (test_stack_init_zero_capacity)
- `tests/test_stack.c:705` (test_stack_verify_valid_stack)
- `tests/test_stack.c:715` (test_stack_verify_null_stack)
- `tests/test_stack.c:722` (test_stack_verify_invalid_api)
- `tests/test_stack.c:735` (test_stack_verify_missing_push_function)
- `tests/test_stack.c:751` (test_stack_verify_missing_pop_function)
- `tests/test_stack.c:767` (test_stack_verify_missing_peek_function)
- `tests/test_stack.c:1170` (test_stack_init_with_array_verify_failure)

3.18.2. REQ-STACK-002: Stack Initial State

Description

The stack shall set length to zero on initialization.

Rationale

Zeroing the length on initialization establishes a known-empty state, ensuring the stack is immediately usable without undefined behavior from uninitialized fields.

Verification Method

Test

Uses (parent)

REQ-SYS-002, REQ-SYS-007, REQ-SYS-008, REQ-SYS-011, REQ-STACK-001

Code Traces

- `src/juno_buff_stack.c:101` (JunoDs_StackInit)

Test Traces

- `tests/test_stack.c:211` (test_stack_init_nominal)

3.18.3. REQ-STACK-003: Stack Push

Description

The stack shall copy an element to the top in O(1) time.

Rationale

O(1) push provides deterministic timing for real-time producers.

Verification Method

Test

Uses (parent)

REQ-STACK-001

Code Traces

- `src/juno_buff_stack.c:27` (JunoDs_StackPush)

Test Traces

- `tests/test_stack.c:248` (test_stack_push_single_item)
- `tests/test_stack.c:265` (test_stack_push_multiple_items)
- `tests/test_stack.c:289` (test_stack_push_to_full_capacity)
- `tests/test_stack.c:330` (test_stack_push_null_stack)
- `tests/test_stack.c:340` (test_stack_push_invalid_pointer)
- `tests/test_stack.c:578` (test_stack_stress_push_pop_cycles)
- `tests/test_stack.c:609` (test_stack_interleaved_operations)

- `tests/test_stack.c:667` (test_stack_fill_empty_cycles)
- `tests/test_stack.c:787` (test_stack_single_element_capacity)
- `tests/test_stack.c:820` (test_stack_data_integrity_after_operations)
- `tests/test_stack.c:848` (test_stack_alternating_push_pop)
- `tests/test_stack.c:966` (test_stack_push_at_exact_capacity)
- `tests/test_stack.c:1079` (test_stack_push_with_setAt_failure)

3.18.4. REQ-STACK-004: Stack Full Rejection

Description

The stack shall return JUNO_STATUS_INVALID_SIZE_ERROR when push is attempted on a full stack.

Rationale

Explicit rejection on a full stack surfaces the overflow condition to the caller rather than silently corrupting adjacent memory.

Verification Method

Test

Uses (parent)

REQ-STACK-003

Code Traces

- `src/juno_buff_stack.c:27` (JunoDs_StackPush)

Test Traces

- `tests/test_stack.c:307` (test_stack_push_overflow)
- `tests/test_stack.c:966` (test_stack_push_at_exact_capacity)

3.18.5. REQ-STACK-005: Stack Pop

Description

The stack shall remove and copy out the top element in O(1) time.

Rationale

O(1) pop provides deterministic timing for real-time consumers.

Verification Method

Test

Uses (parent)

REQ-STACK-001

Code Traces

- `src/juno_buff_stack.c:52` (JunoDs_StackPop)

Test Traces

- `tests/test_stack.c:355` (test_stack_pop_single_item)
- `tests/test_stack.c:378` (test_stack_pop_multiple_items_lifo_order)
- `tests/test_stack.c:420` (test_stack_pop_null_stack)
- `tests/test_stack.c:429` (test_stack_pop_invalid_pointer)
- `tests/test_stack.c:578` (test_stack_stress_push_pop_cycles)
- `tests/test_stack.c:609` (test_stack_interleaved_operations)
- `tests/test_stack.c:667` (test_stack_fill_empty_cycles)
- `tests/test_stack.c:787` (test_stack_single_element_capacity)
- `tests/test_stack.c:820` (test_stack_data_integrity_after_operations)
- `tests/test_stack.c:848` (test_stack_alternating_push_pop)
- `tests/test_stack.c:992` (test_stack_pop_until_empty)
- `tests/test_stack.c:1099` (test_stack_pop_with_getAt_failure)
- `tests/test_stack.c:1148` (test_stack_pop_copy_failure)

3.18.6. REQ-STACK-006: Stack Empty Pop Rejection

Description

The stack shall return JUNO_STATUS_INVALID_SIZE_ERROR when pop is attempted on an empty stack.

Rationale

Explicit rejection on an empty stack surfaces the underflow condition rather than returning stale or uninitialized data.

Verification Method

Test

Uses (parent)

REQ-STACK-005

Code Traces

- `src/juno_buff_stack.c:52` (JunoDs_StackPop)

Test Traces

- `tests/test_stack.c:408` (test_stack_pop_empty_stack)
- `tests/test_stack.c:992` (test_stack_pop_until_empty)

3.18.7. REQ-STACK-007: Stack Peek

Description

The stack shall return a pointer descriptor to the top element without removing it in O(1) time.

Rationale

Peek allows consumers to inspect the top element without consuming it, which is useful for lookahead and conditional pop logic.

Verification Method

Test

Uses (parent)

REQ-STACK-001

Code Traces

- `src/juno_buff_stack.c:77` (JunoDs_StackPeek)

Test Traces

- `tests/test_stack.c:451` (test_stack_peek_single_item)
- `tests/test_stack.c:477` (test_stack_peek_top_element)
- `tests/test_stack.c:501` (test_stack_peek_multiple_times)
- `tests/test_stack.c:538` (test_stack_peek_null_stack)
- `tests/test_stack.c:545` (test_stack_peek_after_pop)
- `tests/test_stack.c:880` (test_stack_peek_consistency_during_operations)
- `tests/test_stack.c:1125` (test_stack_peek_with_getAt_failure)

3.18.8. REQ-STACK-008: Stack Empty Peek Rejection

Description

The stack shall return JUNO_STATUS_INVALID_SIZE_ERROR when peek is attempted on an empty stack.

Rationale

Explicit rejection on empty peek prevents the caller from receiving a descriptor to uninitialized or stale memory.

Verification Method

Test

Uses (parent)

REQ-STACK-007

Code Traces

- `src/juno_buff_stack.c:77` (JunoDs_StackPeek)

Test Traces

- `tests/test_stack.c:528` (test_stack_peek_empty_stack)

3.19. Module: STATUS

3.19.1. REQ-STATUS-001: Unified Status Type

Description

The status module shall define a single 32-bit integer status type (JUNO_STATUS_T) for error propagation.

Rationale

A single, fixed-width status type ensures consistent error propagation across all modules and is portable to any target with stdint.h support.

Verification Method

Inspection

Uses (parent)

REQ-SYS-001, REQ-SYS-003, REQ-SYS-004, REQ-SYS-009

Code Traces

- `include/juno/status.h:50` (<unknown>)

3.19.2. REQ-STATUS-002: Canonical Status Codes

Description

The status module shall define distinct status codes for each error condition category (success, null pointer, memory, type, size, table full, does-not-exist, I/O, CRC, reference, data, timeout, out-of-bounds).

Rationale

Distinct status codes for each error category enable precise diagnosis of failure causes, which is critical on embedded targets where printf-style debugging may be unavailable.

Verification Method

Inspection

Uses (parent)

REQ-SYS-004

Code Traces

- `include/juno/status.h:57` (<unknown>)

3.19.3. REQ-STATUS-003: Failure Handler Callback Type

Description

The status module shall define an optional failure handler callback type that receives a status code, message, and user data.

Rationale

An optional callback type allows integrators to hook diagnostic logging, telemetry, or breakpoint triggers into the error path without modifying library code.

Verification Method

Inspection

Uses (parent)

REQ-SYS-004

Code Traces

- `include/juno/status.h:107` (<unknown>)

3.19.4. REQ-STATUS-004: Failure Handler Non-Interference

Description

The failure handler shall not alter control flow when invoked.

Rationale

Failure handlers must be diagnostic-only to preserve deterministic control flow; allowing handlers to alter execution would introduce unpredictable branching that undermines safety and testability.

Verification Method

Inspection

Uses (parent)

REQ-STATUS-003

Code Traces

- `include/juno/status.h:127` (<unknown>)

3.20. Module: SYS

3.20.1. REQ-SYS-001: Zero Dynamic Allocation

Description

The library shall not use malloc, calloc, realloc, or free.

Rationale

Ensures deterministic behavior with no heap fragmentation or unpredictable timing, enables portability to bare-metal targets that lack a heap, and eliminates heap-related safety defects such as use-after-free, double-free, and memory leaks.

Verification Method

Inspection

Implements (children)

REQ-APP-001, REQ-ARRAY-001, REQ-CRC-001, REQ-HASH-001, REQ-HEAP-001, REQ-IO-001, REQ-LOG-001, REQ-MAP-001, REQ-MATH-001, REQ-MEMORY-001, REQ-MODULE-001, REQ-MTX-001, REQ-POINTER-001, REQ-QUEUE-001, REQ-SB-001, REQ-SCH-001, REQ-SM-001, REQ-STACK-001, REQ-STATUS-001, REQ-TIME-001

Code Traces

- `CMakeLists.txt:75` (list)

3.20.2. REQ-SYS-002: Caller-Owned Memory

Description

All memory used by the library shall be caller-owned and injected via initialization functions.

Rationale

Gives the integrator full control over memory placement (e.g., specific RAM sections), makes memory usage statically analyzable, and avoids hidden coupling between the library and any platform allocator.

Verification Method

Inspection

Implements (children)

REQ-APP-001, REQ-ARRAY-001, REQ-HEAP-001, REQ-IO-001, REQ-LOG-006, REQ-MAP-001, REQ-MEMORY-003, REQ-MTX-001, REQ-QUEUE-001, REQ-QUEUE-002, REQ-SB-003, REQ-SB-005, REQ-SCH-001, REQ-SCH-003, REQ-SM-004, REQ-STACK-001, REQ-STACK-002, REQ-TIME-001

Code Traces

- `include/juno/module.h:54` (JUNO_MODULE_DECLARE)

3.20.3. REQ-SYS-003: Freestanding C11 Compatibility

Description

The library shall compile under -std=c11 -nostdlib -ffreestanding without hosted C standard library dependencies.

Rationale

Targets MCUs, DSPs, and safety-critical platforms that lack a hosted C runtime, while leveraging C11 features such as _Static_assert, _Alignof, and guaranteed fixed-width integer types from stdint.h.

Verification Method

Analysis

Implements (children)

REQ-APP-001, REQ-ARRAY-001, REQ-CRC-001, REQ-HASH-001, REQ-HEAP-001, REQ-IO-001, REQ-LOG-001, REQ-MAP-001, REQ-MATH-001, REQ-MATH-002, REQ-MATH-004, REQ-MEMORY-001, REQ-MODULE-001, REQ-MTX-001, REQ-POINTER-001, REQ-QUEUE-001, REQ-SB-001, REQ-SCH-

001, REQ-SM-001, REQ-STACK-001, REQ-STATUS-001, REQ-TIME-001

Code Traces

- `CMakeLists.txt:75` (list)

3.20.4. REQ-SYS-004: Explicit Status Code Returns

Description

All fallible functions shall return explicit status codes (JUNO_STATUS_T).

Rationale

Provides predictable, deterministic error propagation without platform-dependent mechanisms such as errno, exceptions, or longjmp, which are unavailable or unsafe in freestanding environments.

Verification Method

Inspection

Implements (children)

REQ-HASH-001, REQ-HASH-003, REQ-HEAP-001, REQ-MAP-001, REQ-MEMORY-001, REQ-MODULE-005, REQ-MODULE-006, REQ-POINTER-001, REQ-QUEUE-001, REQ-SB-001, REQ-SM-001, REQ-STACK-001, REQ-STATUS-001, REQ-STATUS-002, REQ-STATUS-003, REQ-TIME-001

Code Traces

- `include/juno/status.h:40` (<unknown>)

3.20.5. REQ-SYS-005: No Silent Failures

Description

The library shall not silently swallow errors or produce undefined behavior on error paths.

Rationale

Aligns with the fundamental assumption that all code is fallible; every error must be surfaced so that integrators can observe, diagnose, and handle failures rather than encountering silent data corruption or undefined behavior.

Verification Method

Inspection

Implements (children)

REQ-CRC-002, REQ-CRC-003, REQ-HASH-001, REQ-HEAP-001, REQ-MAP-001, REQ-MEMORY-001, REQ-POINTER-001, REQ-QUEUE-001, REQ-SB-001, REQ-SM-001, REQ-STACK-001, REQ-TIME-001, REQ-TIME-018

Code Traces

- `include/juno/status.h:40` (<unknown>)

3.20.6. REQ-SYS-006: Dependency Injection via Vtables

Description

All modules shall use the vtable/module-root/derivation pattern for dependency injection.

Rationale

Enables modularity by allowing implementations to be swapped at initialization time, and enables testability by allowing injection of test doubles through the same vtable interface.

Verification Method

Inspection

Implements (children)

REQ-APP-001, REQ-ARRAY-001, REQ-HEAP-001, REQ-IO-001, REQ-IO-009, REQ-IO-013, REQ-LOG-001, REQ-MAP-001, REQ-MEMORY-001, REQ-MODULE-001, REQ-MODULE-003, REQ-MODULE-004, REQ-MTX-001, REQ-POINTER-001, REQ-QUEUE-001, REQ-SB-001, REQ-SCH-001, REQ-SM-001, REQ-SM-002, REQ-STACK-001, REQ-TIME-001, REQ-TIME-002

Code Traces

- `include/juno/module.h:54` (JUNO_MODULE_DECLARE)

3.20.7. REQ-SYS-007: No Global Dependencies

Description

Dependencies shall be injected via initialization functions, never globally referenced.

Rationale

Eliminates hidden coupling between modules, enables multiple independent instances of the same module type, and supports testability by making all dependencies explicit and injectable.

Verification Method

Inspection

Implements (children)

REQ-APP-001, REQ-ARRAY-001, REQ-HEAP-001, REQ-HEAP-002, REQ-IO-001, REQ-LOG-006, REQ-MAP-001, REQ-MAP-002, REQ-MAP-003, REQ-MEMORY-002, REQ-MEMORY-003, REQ-MTX-001, REQ-QUEUE-002, REQ-SB-003, REQ-SCH-001, REQ-SCH-007, REQ-SM-004, REQ-SM-005, REQ-STACK-002, REQ-TIME-001

Code Traces

- `include/juno/macros.h:50` (<unknown>)

3.20.8. REQ-SYS-008: Precondition Verification

Description

All public API functions shall verify their preconditions at entry.

Rationale

Ensures runtime memory safety by failing fast on invalid inputs rather than allowing undefined behavior, which is critical in embedded systems where hardware watchdogs and debuggers may be the only diagnostic tools.

Verification Method

Test

Implements (children)

REQ-ARRAY-005, REQ-ARRAY-007, REQ-HASH-001, REQ-HASH-002, REQ-HEAP-001, REQ-MAP-001, REQ-MEMORY-003, REQ-POINTER-004, REQ-POINTER-005, REQ-QUEUE-002, REQ-SB-003, REQ-SB-012, REQ-SM-009, REQ-SM-011, REQ-STACK-002, REQ-TIME-001, REQ-TIME-019

Code Traces

- `include/juno/macros.h:50` (<unknown>)
- `include/juno/macros.h:88` (<unknown>)

Test Traces

- `tests/test_pointer.c:151` (test_pointer_verify_null_addr)

3.20.9. REQ-SYS-009: No Global Mutable State

Description

The library shall contain no global mutable state.

Rationale

Guarantees reentrancy, avoids hidden coupling between module instances, and enables multiple independent instances to coexist safely in the same address space.

Verification Method

Inspection

Implements (children)

REQ-APP-001, REQ-ARRAY-001, REQ-CRC-001, REQ-HASH-001, REQ-HEAP-001, REQ-IO-001, REQ-LOG-001, REQ-MAP-001, REQ-MATH-001, REQ-MEMORY-001, REQ-MODULE-001, REQ-MTX-001, REQ-POINTER-001, REQ-QUEUE-001, REQ-SB-001, REQ-SCH-001, REQ-SM-001, REQ-STACK-001, REQ-STATUS-001, REQ-TIME-001

Code Traces

- `include/juno/module.h:54` (JUNO_MODULE_DECLARE)

3.20.10. REQ-SYS-010: API Vtable Validation

Description

All modules shall verify that their API vtable and all function pointers within it reference valid (non-NULL) memory before performing any operation.

Rationale

Ensures runtime memory safety by validating dispatch targets before invocation, preventing jumps through NULL or uninitialized function pointers that would cause undefined behavior on bare-metal targets.

Verification Method

Test

Uses (parent)

REQ-SYS-008

Implements (children)

REQ-APP-001, REQ-ARRAY-006, REQ-HEAP-007, REQ-IO-001, REQ-LOG-001, REQ-MAP-009, REQ-MEMORY-001, REQ-MTX-005, REQ-POINTER-004, REQ-QUEUE-001, REQ-SB-010, REQ-SCH-001, REQ-SM-001, REQ-SM-010, REQ-STACK-001, REQ-TIME-001

Code Traces

- `include/juno/macros.h:50` (<unknown>)

Test Traces

- `tests/test_array.c:238` (test_array_verify_null_api)

3.20.11. REQ-SYS-011: Injected Dependency Validation

Description

All modules shall verify that all injected dependency pointers are valid (non-NULL) before performing any operation.

Rationale

Ensures runtime memory safety by validating injected dependencies before use, catching misconfigured or partially initialized module graphs before they produce undefined behavior.

Verification Method

Test

Uses (parent)

REQ-SYS-008

Implements (children)

REQ-HEAP-008, REQ-MAP-010, REQ-MEMORY-003, REQ-QUEUE-002, REQ-SB-010, REQ-SB-011, REQ-SM-010, REQ-STACK-002, REQ-TIME-001

Code Traces

- `include/juno/macros.h:50` (<unknown>)

Test Traces

- `tests/test_sb.c:346` (test_sb_broker_verify_null_registry)

3.21. Module: TIME

3.21.1. REQ-TIME-001: Timestamp Representation

Description

The time module shall represent time as a struct with integer seconds and fixed-point fractional subseconds fields.

Rationale

Integer seconds with fixed-point subseconds provides deterministic, lossless time representation without floating-point rounding, which is essential for real-time scheduling and time-triggered architectures.

Verification Method

Inspection

Uses (parent)

REQ-SYS-001, REQ-SYS-002, REQ-SYS-003, REQ-SYS-004, REQ-SYS-005, REQ-SYS-006, REQ-SYS-007, REQ-SYS-008, REQ-SYS-009, REQ-SYS-010, REQ-SYS-011

Code Traces

- `include/juno/time/time_api.h:104` (<unknown>)

3.21.2. REQ-TIME-002: Time API Interface

Description

The time module shall define a vtable-based API with Now, AddTime, SubtractTime, SleepTo, Sleep, and conversion operations.

Rationale

A vtable-based time API decouples the scheduler and application logic from platform-specific clock hardware, enabling both real hardware clocks and deterministic test clocks.

Verification Method

Inspection

Uses (parent)

REQ-SYS-006

Code Traces

- `include/juno/time/time_api.h:128` (JUNO_TIMESTAMP_RESULT_T)

3.21.3. REQ-TIME-003: AddTime Operation

Description

The time module shall add a duration to a timestamp in-place.

Rationale

In-place addition avoids allocating a new timestamp, and is the fundamental operation for computing deadlines and advancing time.

Verification Method

Test

Uses (parent)

REQ-TIME-002

Code Traces

- `src/juno_time.c:26` (JunoTime_AddTime)

Test Traces

- `tests/test_time.c:86` (test_AddTime_no_subseconds_overflow)

3.21.4. REQ-TIME-004: AddTime Subsecond Carry-Over

Description

The time module shall carry over subseconds to seconds when subsecond addition overflows.

Rationale

Correct carry-over from subseconds to seconds is essential for accurate time arithmetic; without it, subsecond accumulation would overflow and produce incorrect timestamps.

Verification Method

Test

Uses (parent)

REQ-TIME-003

Code Traces

- `src/juno_time.c:26` (JunoTime_AddTime)

Test Traces

- `tests/test_time.c:97` (test_AddTime_with_subseconds_overflow)

3.21.5. REQ-TIME-005: SubtractTime Operation

Description

The time module shall subtract a duration from a timestamp in-place.

Rationale

In-place subtraction is the fundamental operation for computing elapsed time and remaining durations.

Verification Method

Test

Uses (parent)

REQ-TIME-002

Code Traces

- `src/juno_time.c:49` (JunoTime_SubtractTime)

Test Traces

- `tests/test_time.c:110` (test_SubtractTime_no_borrow)

3.21.6. REQ-TIME-006: SubtractTime Subsecond Borrow

Description

The time module shall borrow from seconds when subsecond subtraction underflows.

Rationale

Correct borrow from seconds to subseconds is essential for accurate time subtraction; without it, subsecond underflow would produce incorrect timestamps.

Verification Method

Test

Uses (parent)

REQ-TIME-005

Code Traces

- `src/juno_time.c:49` (JunoTime_SubtractTime)

Test Traces

- `tests/test_time.c:122` (test_SubtractTime_borrow_subseconds)

3.21.7. REQ-TIME-007: SubtractTime Negative Saturation

Description

The time module shall saturate the result to zero seconds and zero subseconds when subtraction would produce negative time.

Rationale

Saturating to zero prevents negative time values, which have no physical meaning in the library's time model and would cause undefined behavior in unsigned arithmetic.

Verification Method

Test

Uses (parent)

REQ-TIME-005

Code Traces

- `src/juno_time.c:49` (JunoTime_SubtractTime)

Test Traces

- `tests/test_time.c:135` (test_SubtractTime_invalid_lower_seconds)
- `tests/test_time.c:146` (test_SubtractTime_invalid_zero_seconds_insufficient_subseconds)

3.21.8. REQ-TIME-008: SubtractTime Negative Error

Description

The time module shall return JUNO_STATUS_INVALID_DATA_ERROR when subtraction would produce negative time.

Rationale

Returning an error status when saturation occurs allows callers to detect and handle the underflow condition rather than silently receiving a clamped result.

Verification Method

Test

Uses (parent)

REQ-TIME-007

Code Traces

- `src/juno_time.c:49` (JunoTime_SubtractTime)

Test Traces

- `tests/test_time.c:135` (test_SubtractTime_invalid_lower_seconds)
- `tests/test_time.c:146` (test_SubtractTime_invalid_zero_seconds_insufficient_subseconds)

3.21.9. REQ-TIME-009: Timestamp to Nanoseconds Conversion

Description

The time module shall convert a timestamp to nanoseconds using half-up rounding.

Rationale

Nanosecond conversion enables interfacing with platform APIs and POSIX timespec structures that use nanosecond resolution.

Verification Method

Test

Uses (parent)

REQ-TIME-002

Code Traces

- `src/juno_time.c:78` (JunoTime_TimestampToNanos)

Test Traces

- `tests/test_time.c:157` (test_TimestampToNanos_success_integer)

- `tests/test_time.c:166` (test_TimestampToNanos_success_fractional)

3.21.10. REQ-TIME-010: Timestamp to Microseconds Conversion

Description

The time module shall convert a timestamp to microseconds using half-up rounding.

Rationale

Microsecond conversion enables interfacing with platform timers and IO timeout APIs that use microsecond resolution.

Verification Method

Test

Uses (parent)

REQ-TIME-002

Code Traces

- `src/juno_time.c:115` (JunoTime_TimestampToMicros)

Test Traces

- `tests/test_time.c:177` (test_TimestampToMicros_success_integer)
- `tests/test_time.c:186` (test_TimestampToMicros_success_fractional)

3.21.11. REQ-TIME-011: Timestamp to Milliseconds Conversion

Description

The time module shall convert a timestamp to milliseconds using half-up rounding.

Rationale

Millisecond conversion enables interfacing with RTOS tick-based APIs and human-readable timing displays.

Verification Method

Test

Uses (parent)

REQ-TIME-002

Code Traces

- `src/juno_time.c:149` (JunoTime_TimestampToMillis)

Test Traces

- `tests/test_time.c:196` (test_TimestampToMillis_success_integer)
- `tests/test_time.c:205` (test_TimestampToMillis_success_fractional)

3.21.12. REQ-TIME-012: Nanoseconds to Timestamp Conversion

Description

The time module shall convert nanoseconds to a timestamp with integer seconds and fractional subseconds.

Rationale

Conversion from nanoseconds enables constructing timestamps from platform clock reads and POSIX timespec values.

Verification Method

Test

Uses (parent)

REQ-TIME-002

Code Traces

- `src/juno_time.c:184` (JunoTime_NanosToTimestamp)

Test Traces

- `tests/test_time.c:215` (test_NanosToTimestamp_zero)
- `tests/test_time.c:225` (test_NanosToTimestamp_integer_and_fractional)

3.21.13. REQ-TIME-013: Microseconds to Timestamp Conversion

Description

The time module shall convert microseconds to a timestamp with integer seconds and fractional subseconds.

Rationale

Conversion from microseconds enables constructing timestamps from timer registers and IO timeout values.

Verification Method

Test

Uses (parent)

REQ-TIME-002

Code Traces

- `src/juno_time.c:202` (JunoTime_MicrosToTimestamp)

Test Traces

- `tests/test_time.c:240` (test_MicrosToTimestamp_zero)
- `tests/test_time.c:250` (test_MicrosToTimestamp_integer_and_fractional)

3.21.14. REQ-TIME-014: Milliseconds to Timestamp Conversion

Description

The time module shall convert milliseconds to a timestamp with integer seconds and fractional subseconds.

Rationale

Conversion from milliseconds enables constructing timestamps from RTOS tick counts and human-specified delays.

Verification Method

Test

Uses (parent)

REQ-TIME-002

Code Traces

- `src/juno_time.c:220` (JunoTime_MillisToTimestamp)

Test Traces

- `tests/test_time.c:265` (test_MillisToTimestamp_zero)
- `tests/test_time.c:275` (test_MillisToTimestamp_integer_and_fractional)

3.21.15. REQ-TIME-015: Timestamp to Double Conversion

Description

The time module shall convert a timestamp to a double-precision floating-point seconds value.

Rationale

Double-precision conversion provides a convenient floating-point representation for algorithms that require fractional seconds, such as control loops and telemetry formatting.

Verification Method

Test

Uses (parent)

REQ-TIME-002

Code Traces

- `src/juno_time.c:239` (JunoTime_TimestampToDouble)

Test Traces

- `tests/test_time.c:314` (test_timestamp_to_double)

3.21.16. REQ-TIME-016: Double to Timestamp Conversion

Description

The time module shall convert a non-negative double-precision seconds value to a timestamp.

Rationale

Conversion from double enables constructing timestamps from floating-point time sources and configuration values.

Verification Method

Test

Uses (parent)

REQ-TIME-002

Code Traces

- `src/juno_time.c:254` (JunoTime_DoubleToTimestamp)

Test Traces

- `tests/test_time.c:324` (test_double_to_timestamp_success)
- `tests/test_time.c:333` (test_double_to_timestamp_zero)

3.21.17. REQ-TIME-017: Double to Timestamp Range Rejection

Description

The time module shall return JUNO_STATUS_INVALID_DATA_ERROR for negative or out-of-range double inputs.

Rationale

Rejecting negative and out-of-range double inputs prevents construction of timestamps with undefined or physically meaningless values.

Verification Method

Test

Uses (parent)

REQ-TIME-016

Code Traces

- `src/juno_time.c:254` (JunoTime_DoubleToTimestamp)

Test Traces

- `tests/test_time.c:342` (test_double_to_timestamp_negative_rejected)

3.21.18. REQ-TIME-018: Conversion Overflow Detection

Description

The time module shall return JUNO_STATUS_INVALID_DATA_ERROR when a timestamp-to-integer conversion would overflow.

Rationale

Detecting overflow before it occurs prevents silent truncation or wraparound, which would produce incorrect timing values in downstream consumers.

Verification Method

Test

Uses (parent)

REQ-SYS-005

Code Traces

- `src/juno_time.c:78` (JunoTime_TimestampToNanos)
- `src/juno_time.c:115` (JunoTime_TimestampToMicros)
- `src/juno_time.c:149` (JunoTime_TimestampToMillis)

Test Traces

- `tests/test_time.c:349` (test_timestamp_to_nanos_overflow)

3.21.19. REQ-TIME-019: Null Pointer Rejection

Description

The time module shall return JUNO_STATUS_NULLPTR_ERROR when the time root pointer is NULL.

Rationale

Rejecting a NULL time root pointer prevents null dereference, ensuring runtime memory safety at the module entry point.

Verification Method

Test

Uses (parent)

REQ-SYS-008

Code Traces

- `src/juno_time.c:78` (JunoTime_TimestampToNanos)
- `src/juno_time.c:115` (JunoTime_TimestampToMicros)
- `src/juno_time.c:149` (JunoTime_TimestampToMillis)
- `src/juno_time.c:184` (JunoTime_NanosToTimestamp)
- `src/juno_time.c:202` (JunoTime_MicrosToTimestamp)
- `src/juno_time.c:220` (JunoTime_MillisToTimestamp)
- `src/juno_time.c:239` (JunoTime_TimestampToDouble)
- `src/juno_time.c:254` (JunoTime_DoubleToTimestamp)

Test Traces

- `tests/test_time.c:362` (test_double_to_timestamp_null_root)

4. Traceability Validation Summary

 All traceability checks passed.